Uninformed Search

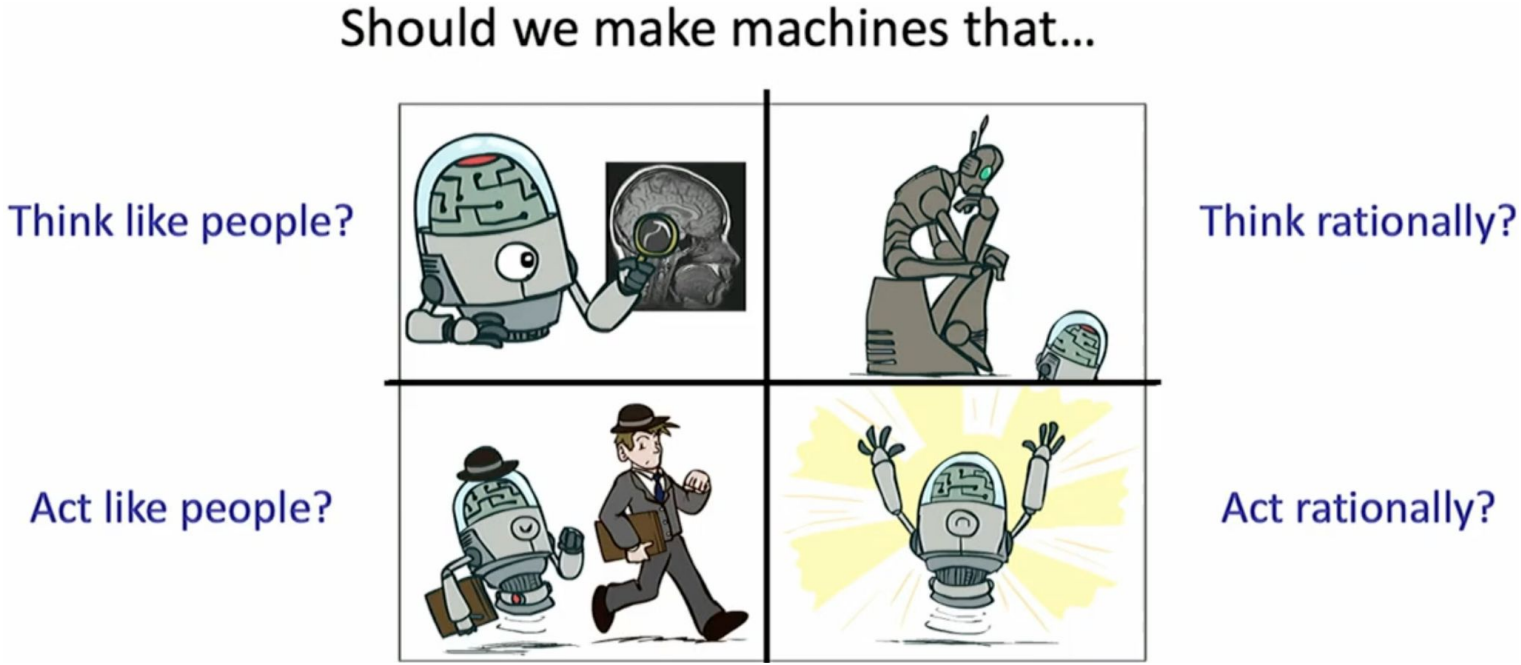
Arash Marioriyad

5 Esfand 1404

CE 417-Artificial Intelligence
Sharif Univ. of Tech.

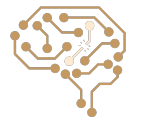
Some of Slides have been adopted from Berkeley CS188

What should we build?



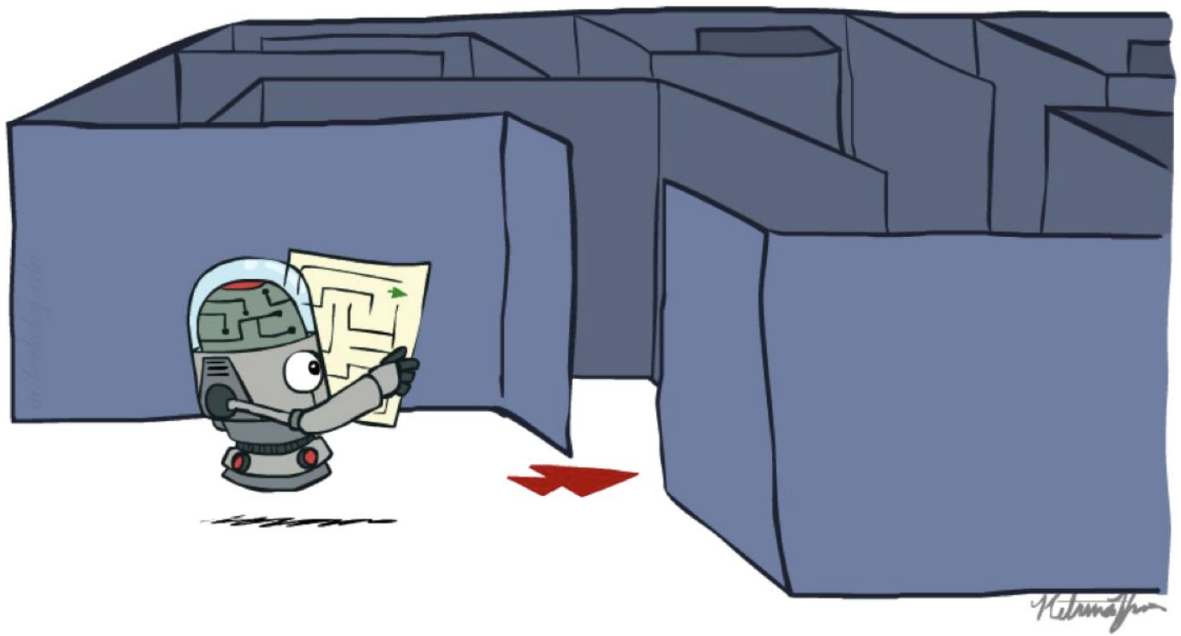
Rational Decisions

- Rational: maximally achieving pre-defined goals
- Goals are expressed in terms of the utility of outcomes
- World is uncertain, so we'll use expected utility
- Being rational = acting to **maximize your expected utility**



Today

Search

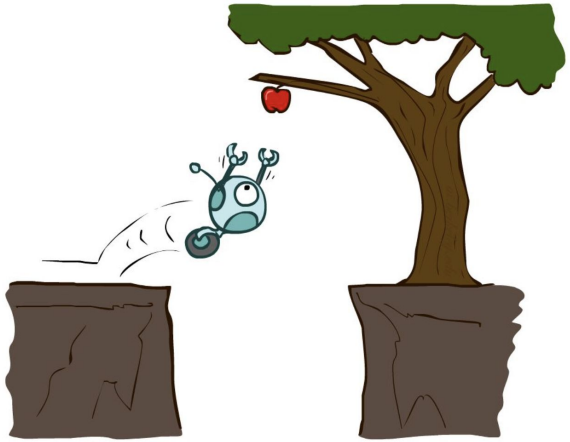


Why Do We Care About Search?

- The algorithms we'll discuss are incredibly widely applied
  - Pathing / Routing Problems
  - Chip Design (Routing Wires)
  - Resource Planning Problems
  - Scheduling Problems
  - Protein Design
  - Video Games

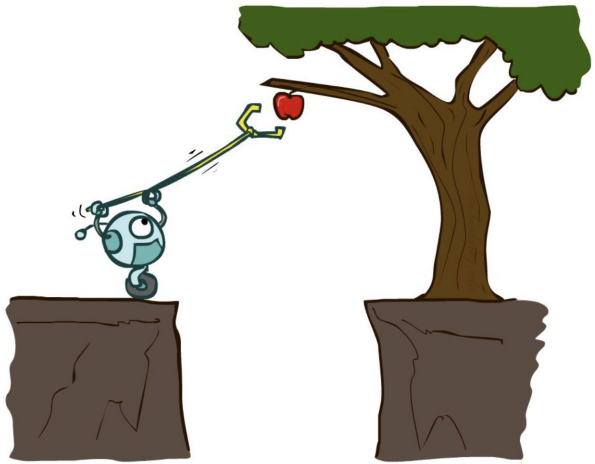
Reflex Agents

- Choose action based on current percept
- Do not consider the future consequences of their actions
- Consider how the world IS
- Can a reflex agent be rational?



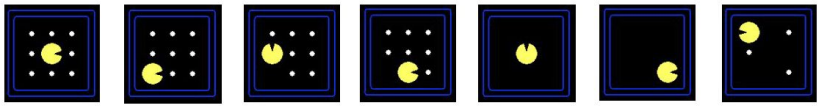
Planning Agents

- Ask “what if”
- Decisions based on (hypothesized) consequences of actions
- Must have a model of how the world evolves in response to actions
- Must formulate a goal (test)
- Consider how the world WOULD BE

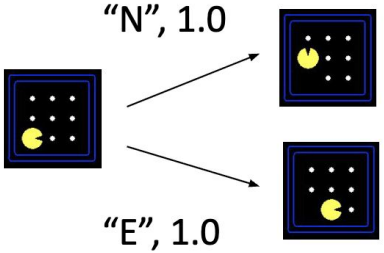


Search Problems

- A search problem consists of:
  - A state space



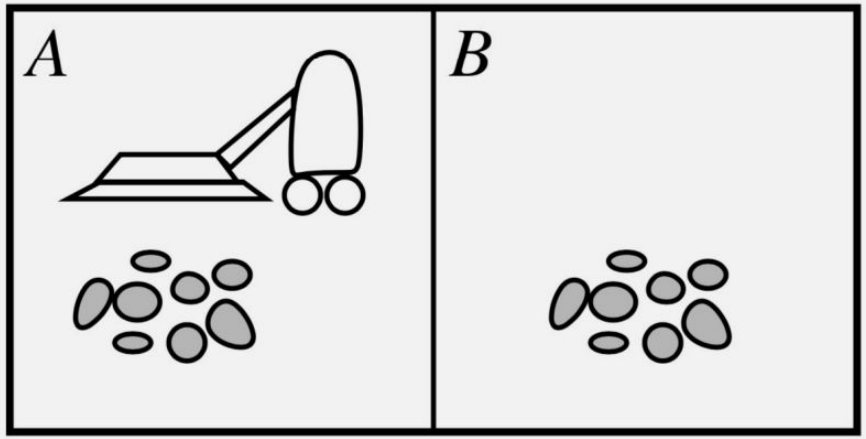
  - A successor function (with actions, costs)



  - A start state and a goal test

- A solution is a sequence of actions (a plan) which transforms the start state to a goal state

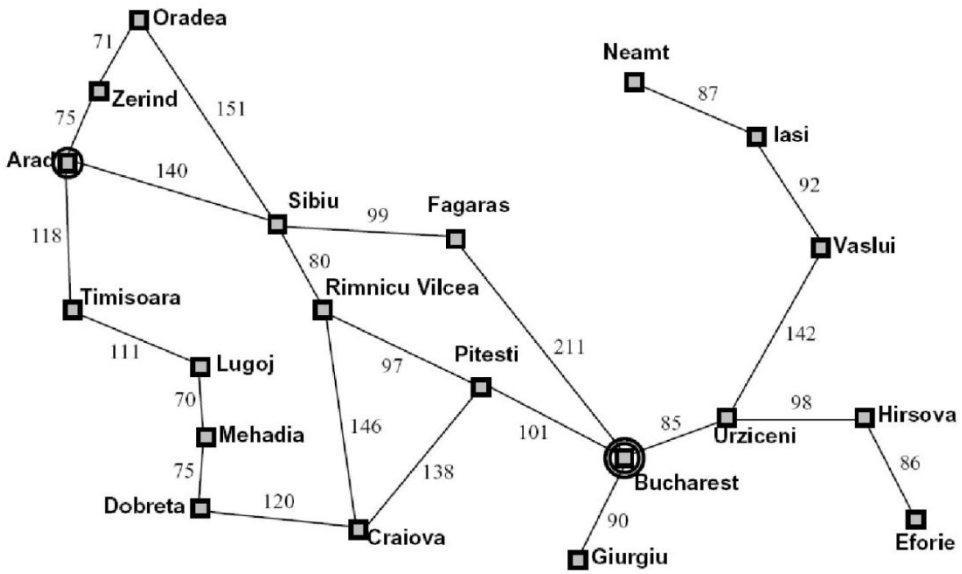
Vacuum-cleaner Toy Example



Percepts: location and contents, e.g., $[A, Dirty]$

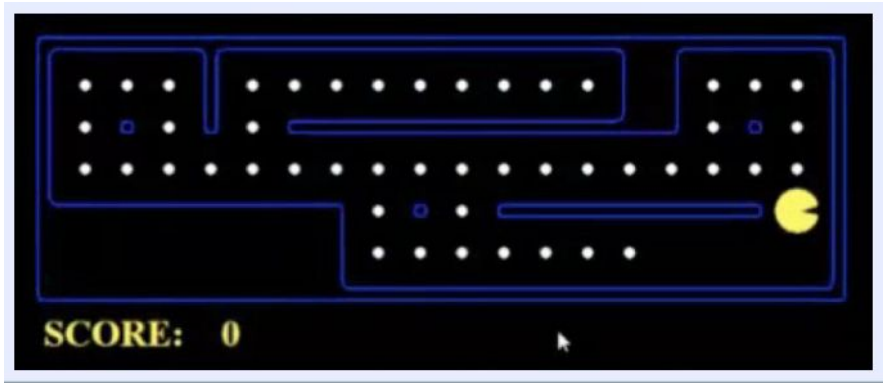
Actions: $Left$, $Right$, $Suck$, $NoOp$

Example: Traveling in Romania



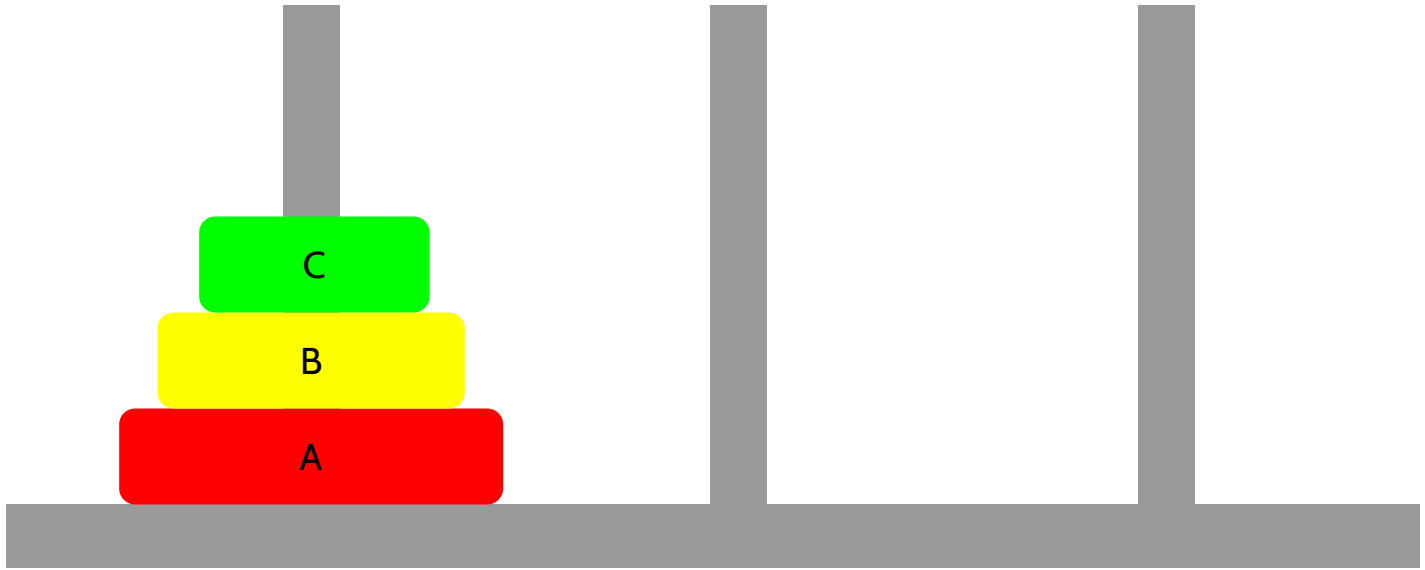
- State space:
  - Cities
- Successor function:
  - Roads: Go to adjacent city with cost = distance
- Start state:
  - Arad
- Goal test:
  - Is state == Bucharest?
- Solution?

Example: Pacman

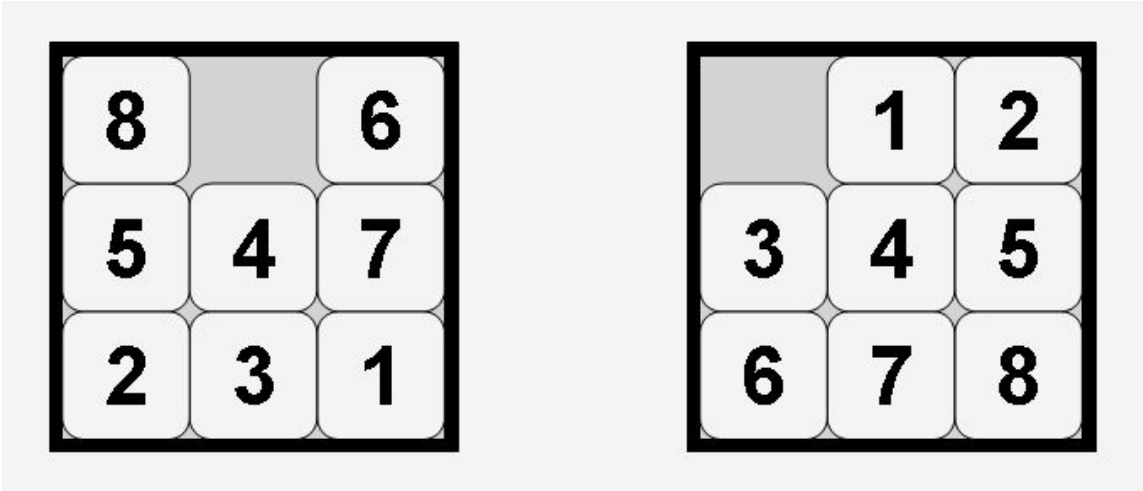


- Problem: Eat-All-Dots
  - States: {(x,y), dot booleans}
  - Actions: NSEW
  - Successor: update location and possibly a dot boolean
  - Goal test: dots all false

Towers of Hanoi

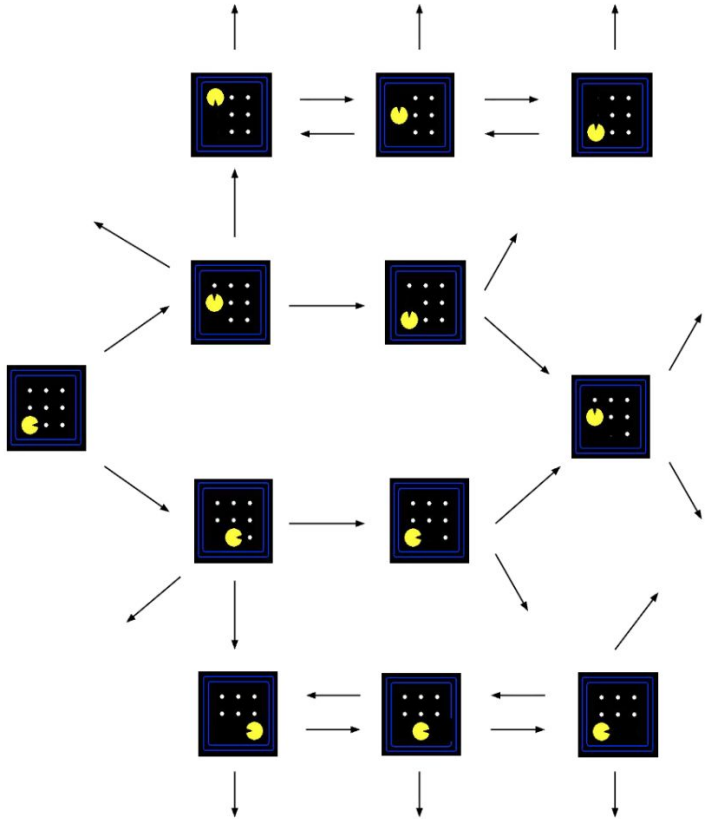


Eight Puzzle

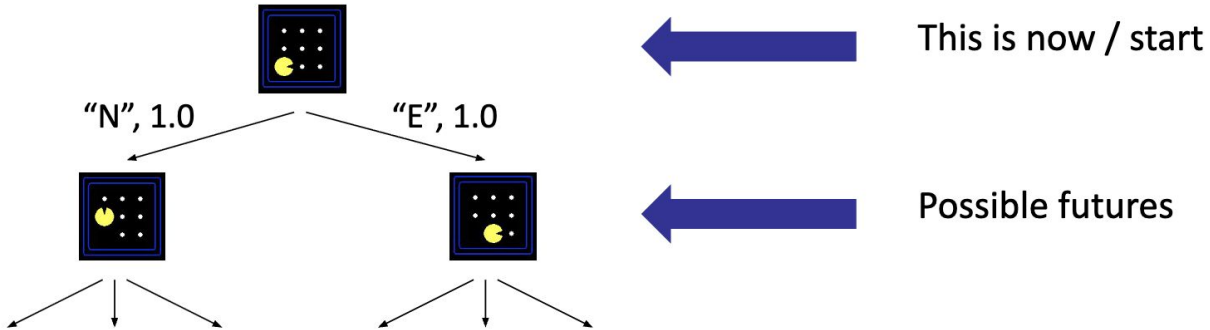


State Space Graph

- State space graph: A mathematical representation of a search problem
  - Nodes are (abstracted) world configurations
  - Arcs represent successors (action results)
  - The goal test is a set of goal nodes (maybe only one)
- In a state space graph, each state occurs only once!
- We can rarely build this full graph in memory (it's too big), but it's a useful idea



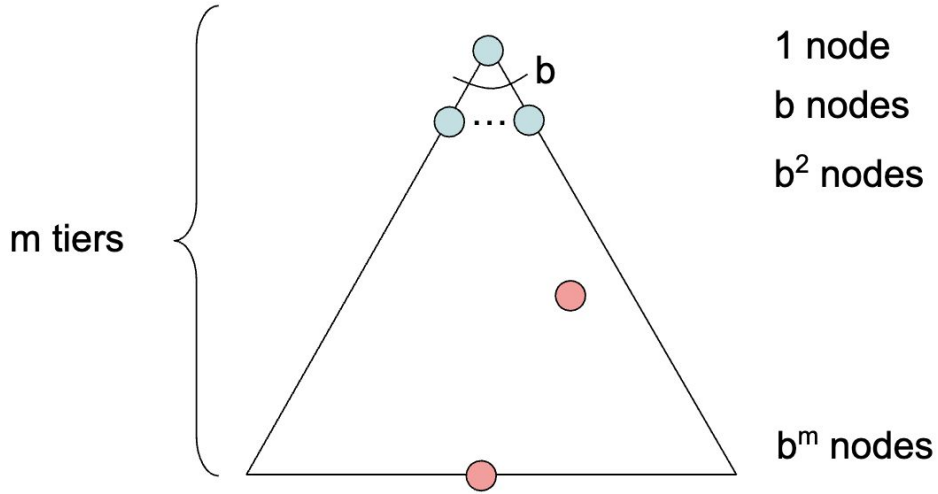
Search Tree



- A search tree:
  - A "what if" tree of plans and their outcomes
  - The start state is the root node
  - Children correspond to successors
  - Nodes show states, but correspond to PLANS that achieve those states
  - For most problems, we can never actually build the whole tree

About Search Trees

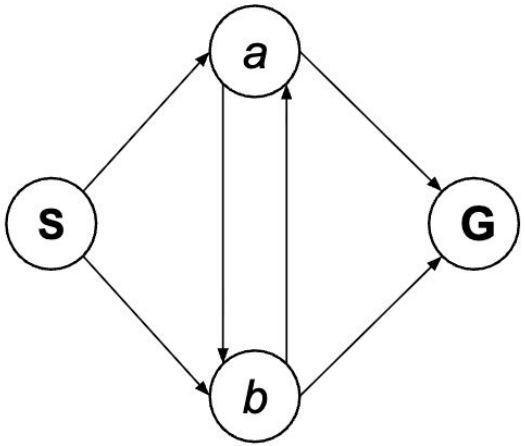
- b is the branching factor
- m is the maximum depth
- solutions at various depths
- Number of nodes in entire tree?
  - $1 + b + b^2 + \ldots + b^m = O(b^m)$



Quiz

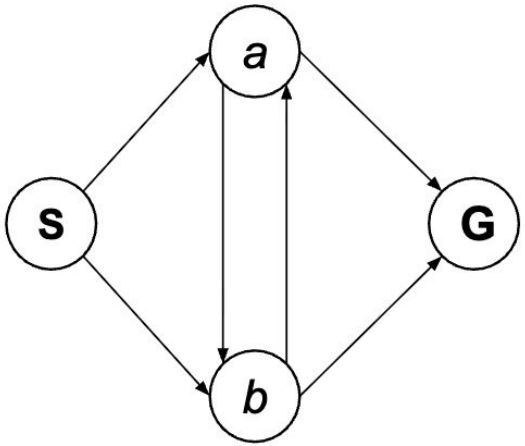
Consider this 4-state graph:

How big is its search tree (from S)?

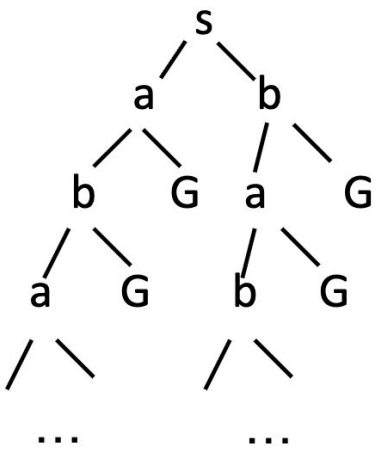


Quiz Solution

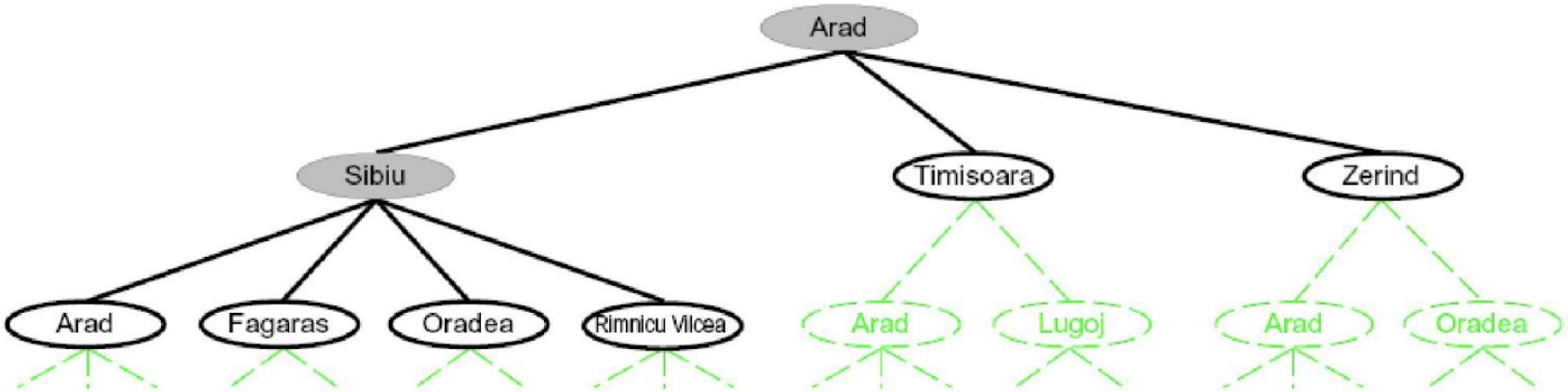
Consider this 4-state graph:



How big is its search tree (from S)?



Tree Search

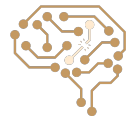


- Search:
  - Expand out potential plans (tree nodes)
  - Maintain a fringe of partial plans under consideration
  - Try to expand as few tree nodes as possible

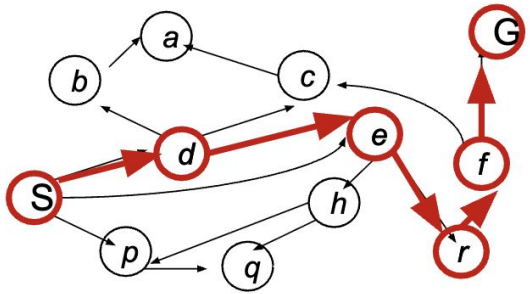
General Tree Search

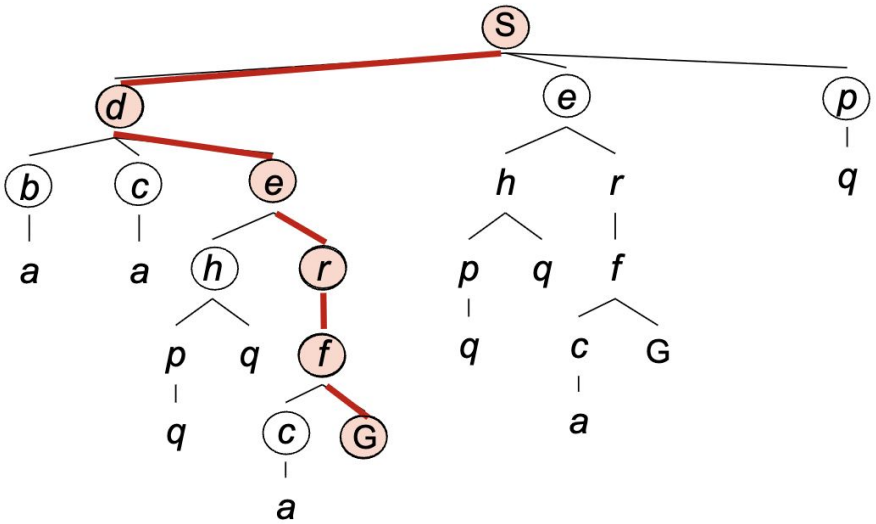
```
function TREE-SEARCH( problem, strategy) returns a solution, or failure
   initialize the search tree using the initial state of problem
   loop do
      if there are no candidates for expansion then return failure
      choose a leaf node for expansion according to strategy
      if the node contains a goal state then return the corresponding solution
      else expand the node and add the resulting nodes to the search tree
   end
```

- Important ideas:
  - Fringe
  - Expansion
  - Exploration strategy
- Main question: which fringe nodes to explore?



Example: Tree Search





~~s~~

~~s □ d~~

s □ e

s □ p

s □ d □ b

s □ d □ c

~~s □ d □ e~~

s □ d □ e □ h

~~s □ d □ e □ r~~

~~s □ d □ e □ r □ f~~

s □ d □ e □ r □ f □ c

~~s □ d □ e □ r □ f □ G~~

What Matters in Search Algorithms?

- Completeness: Guaranteed to find a solution if one exists?
- Optimality: Guaranteed to find the least cost path?
- Time Complexity
- Space Complexity

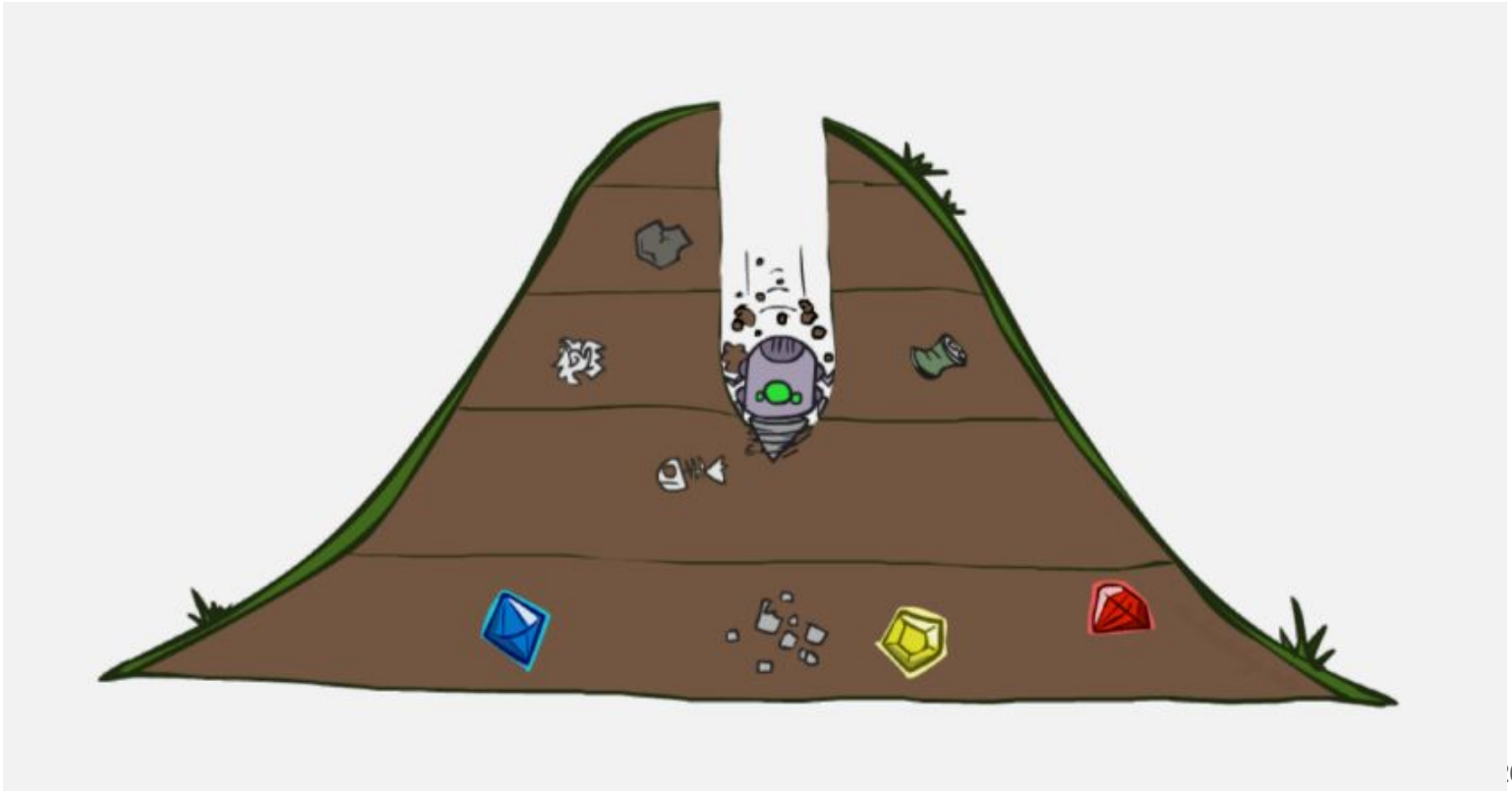
Search Algorithms

- **Uninformed (Blind) Search:**
  - No additional information about states beyond the problem definition (start, goal, successors)
- **Informed (Heuristic) Search:**
  - Uses the additional information about the estimated cost to goal

Uninformed Search

- Depth-First Search (DFS)
- Breadth-First Search (BFS)
- Iterative Deepening Search (IDS)
- Uniform-Cost Search (UCS)

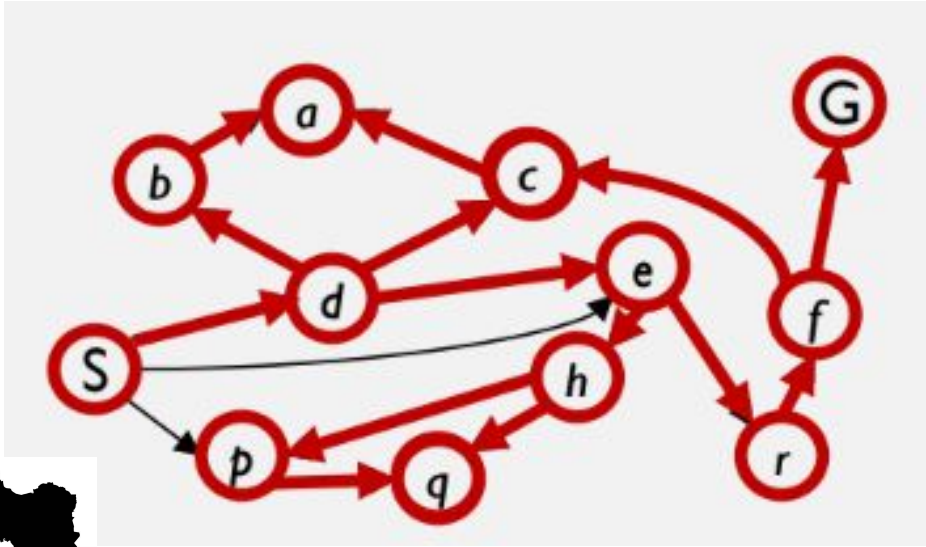
Depth-First Search

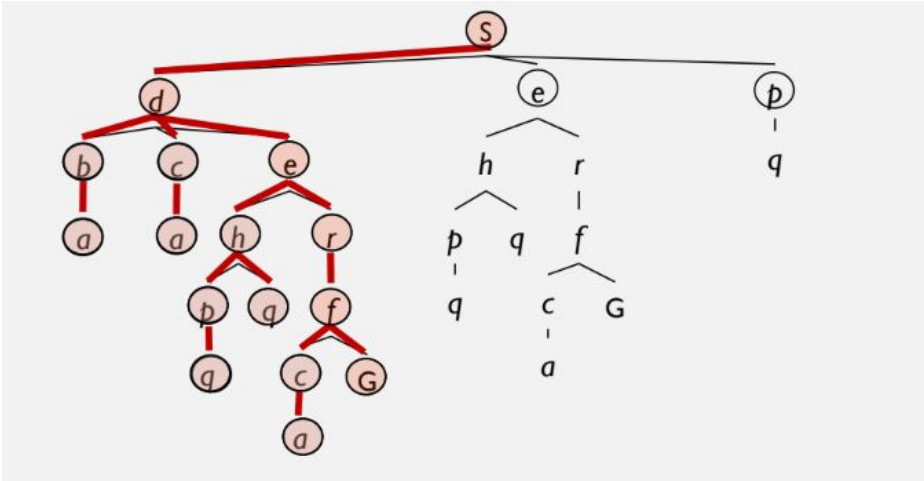


Depth-First Search: Expand the Deepest Node First

Strategy: expand a deepest node first

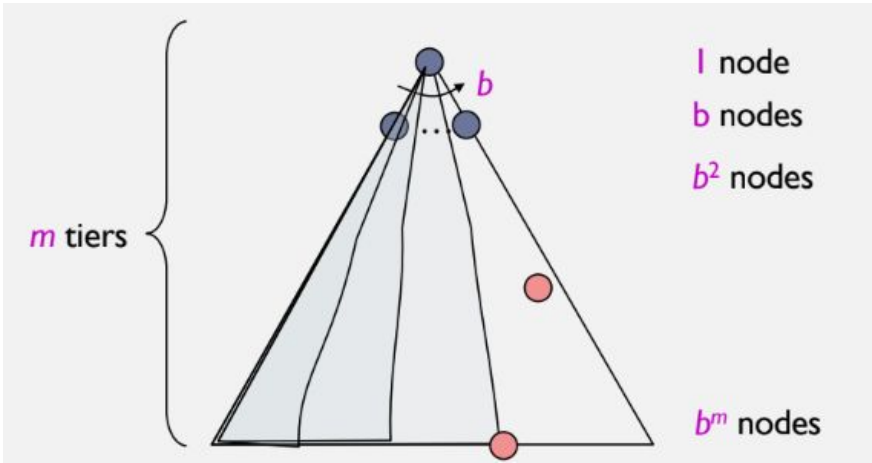
Implementation: Frontier is a LIFO stack



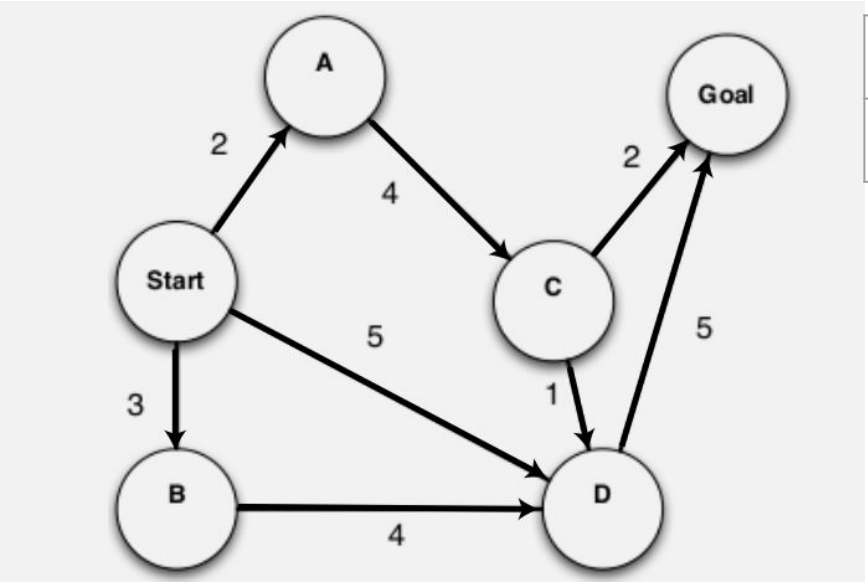


Depth-First Search (DFS) Properties

- Time Complexity
  - Some left prefix of the tree down to depth m.
  - Could process the whole tree!
  - If m is finite, takes time **O(b^m)**
- Space Complexity
  - Only has siblings on path to root, so **O(bm)**
- Is it complete?
  - No, m could be infinite
- Is it optimal?
  - No, it finds the “leftmost” solution, regardless of depth or cost

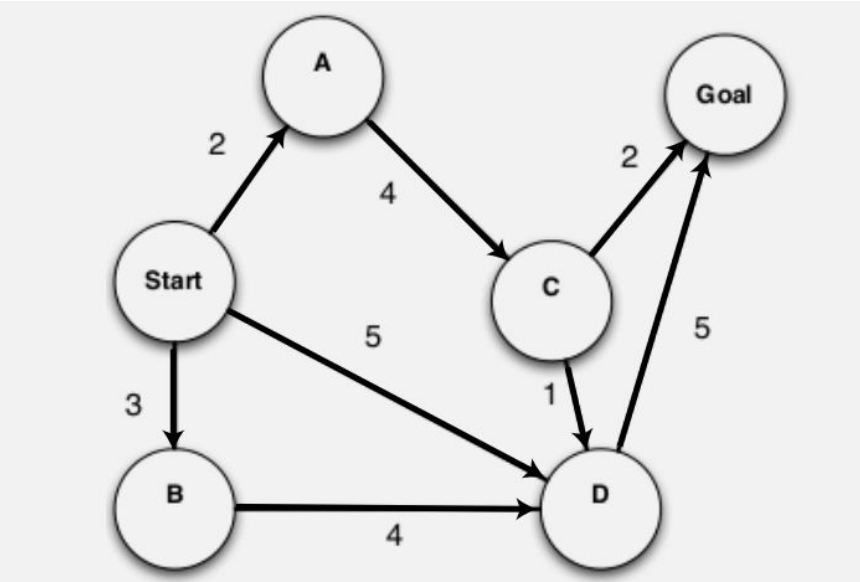


Tree Search Algorithms - DFS



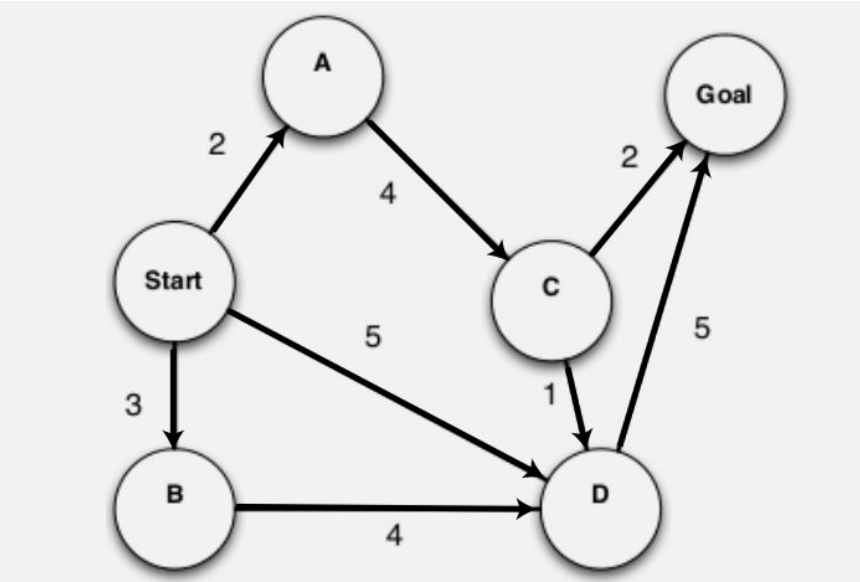
step	Frontiers/Fringe
1	**S**

Tree Search Algorithms - DFS



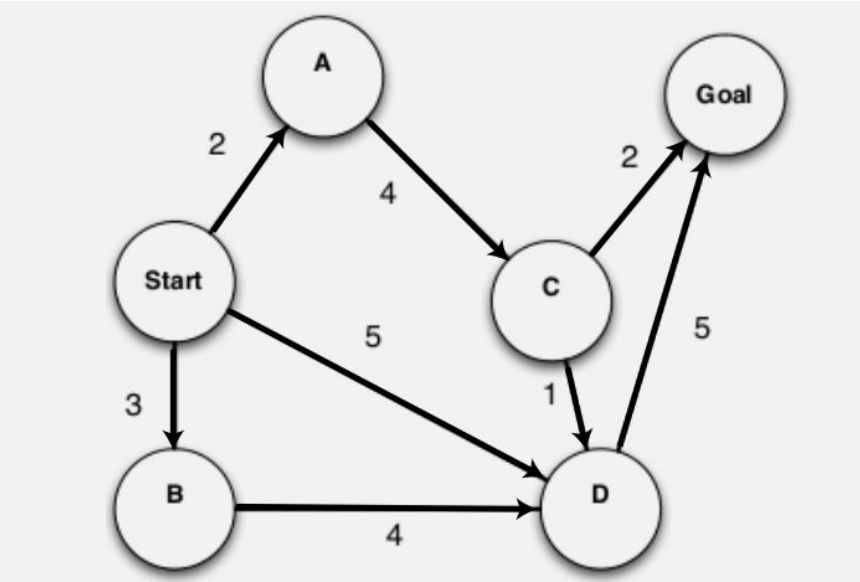
step	Frontiers/Fringe
1	S
2	~~S~~, (S D), (S B), **(S A)**

Tree Search Algorithms - DFS



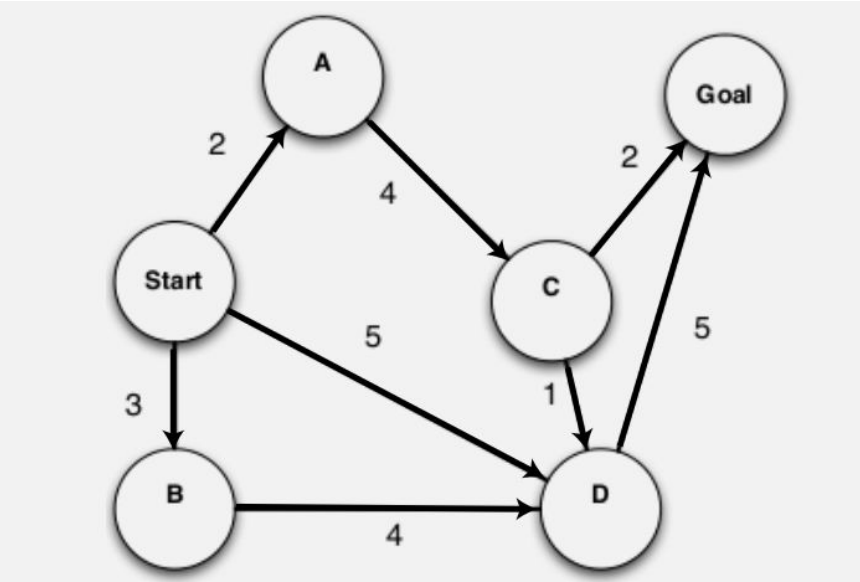
step	Frontiers/Fringe
1	S
2	~~S~~, (S D), (S B), **(S A)**
3	~~S~~, (S D), (S B), ~~(S A)~~, **(S A C)**

Tree Search Algorithms - DFS



step	Frontiers/Fringe
1	S
2	~~S~~, (S D), (S B), **(S A)**
3	~~S~~, (S D), (S B), ~~(S A)~~, ~~(S A C)~~, (S A C G), **(S A C D)**

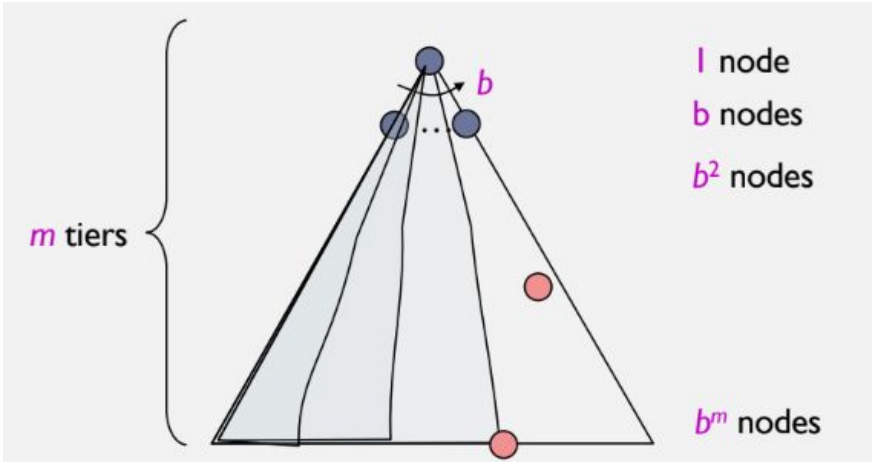
Tree Search Algorithms - DFS



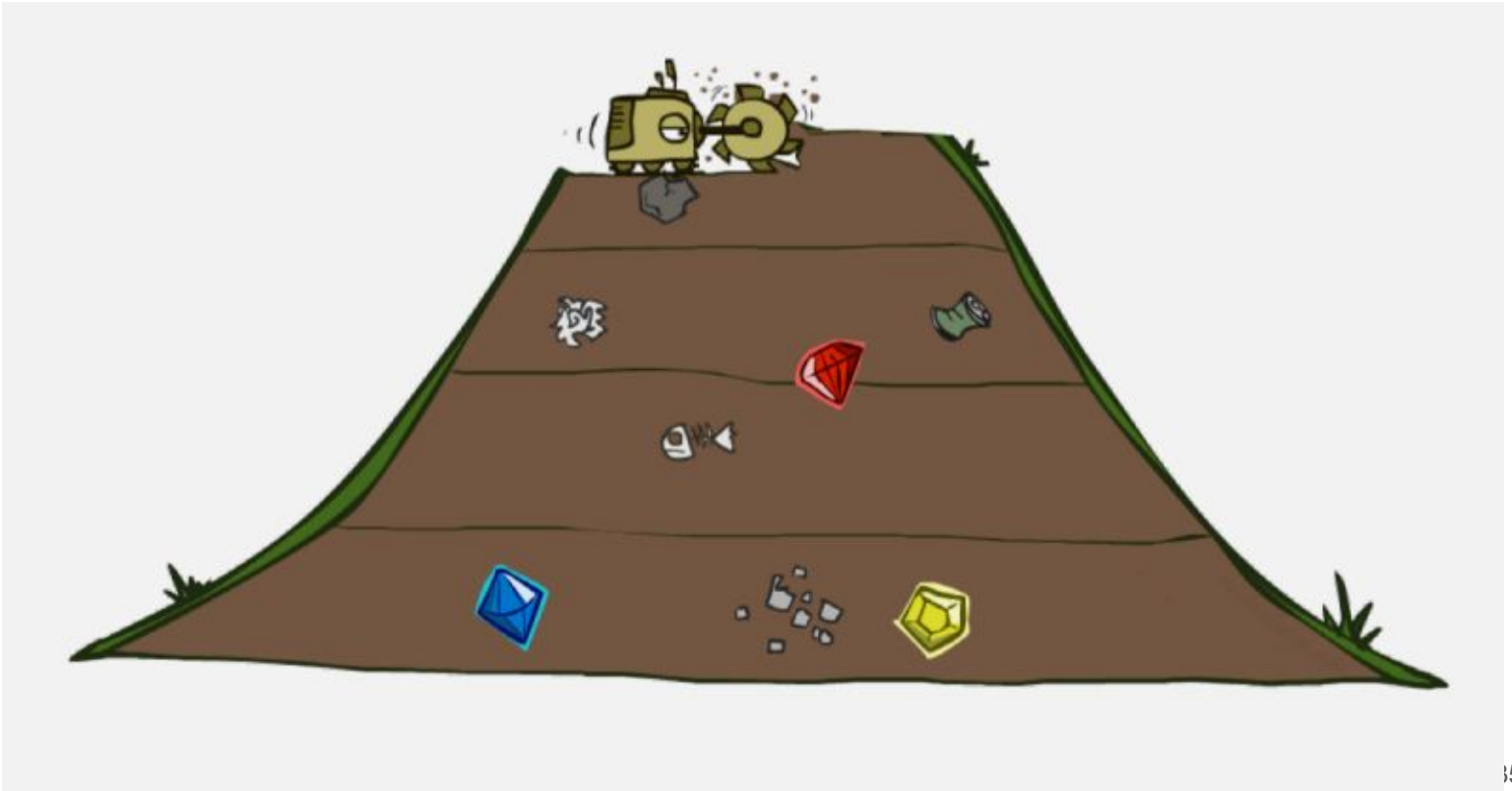
step	Frontiers/Fringe
1	S
2	~~S~~, (S D), (S B), **(S A)**
3	~~S~~, (S D), (S B), ~~(S A)~~, ~~(S A C)~~, (S A C G), (S A C D)
3	~~S~~, (S D), (S B), ~~(S A)~~, ~~(S A C)~~, (S A C G), ~~(S A C D~~**), (S A C D G)**

Depth-First Search (DFS) Properties

- Time Complexity
  - Some left prefix of the tree down to depth m.
  - Could process the whole tree!
  - If m is finite, takes time **O(b^m)**
- Space Complexity
  - Only has siblings on path to root, so **O(bm)**
- Is it complete?
  - No, m could be infinite
- Is it optimal?
  - No, it finds the “leftmost” solution, regardless of depth or cost



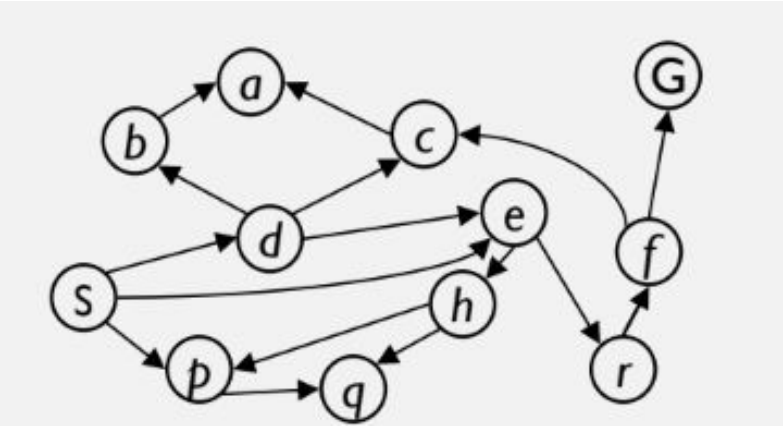
Breadth-First Search

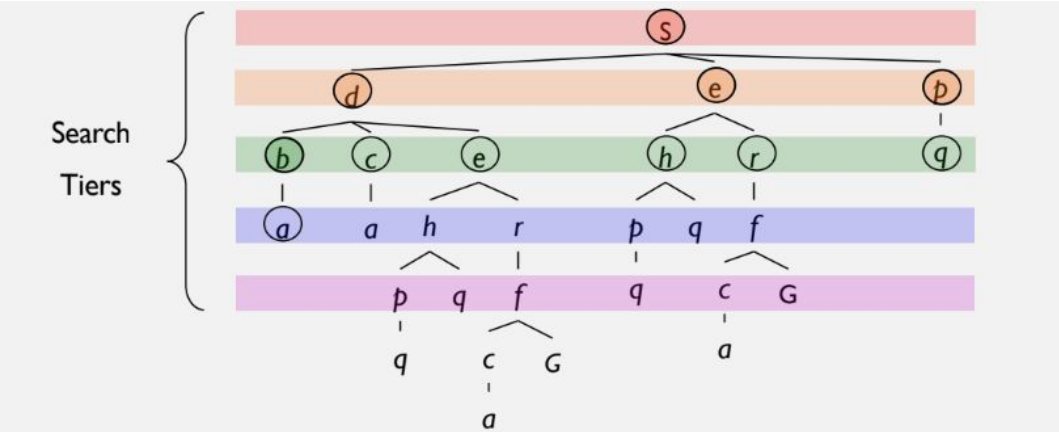


Breadth-First Search: Expand the Shallowest Node First

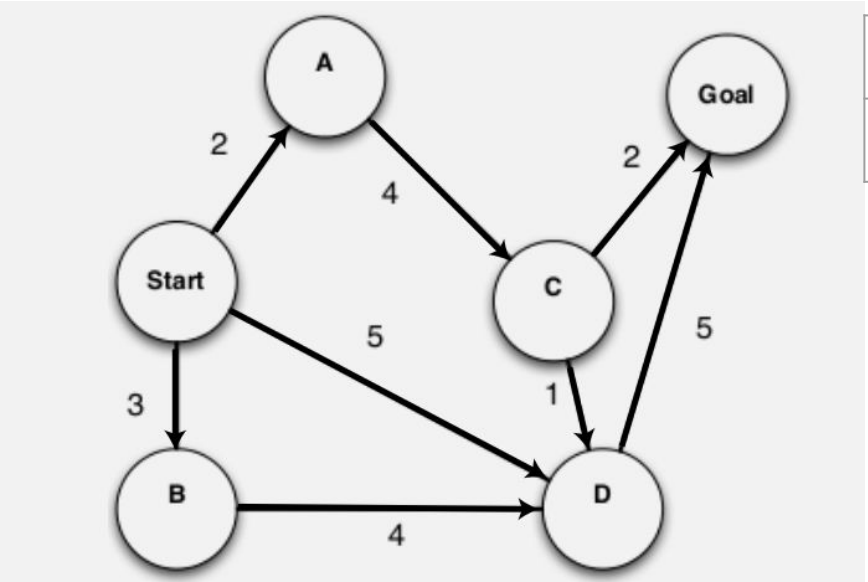
Strategy: expand a shallowest node first

Implementation: Frontier (Fringe) is a FIFO queue



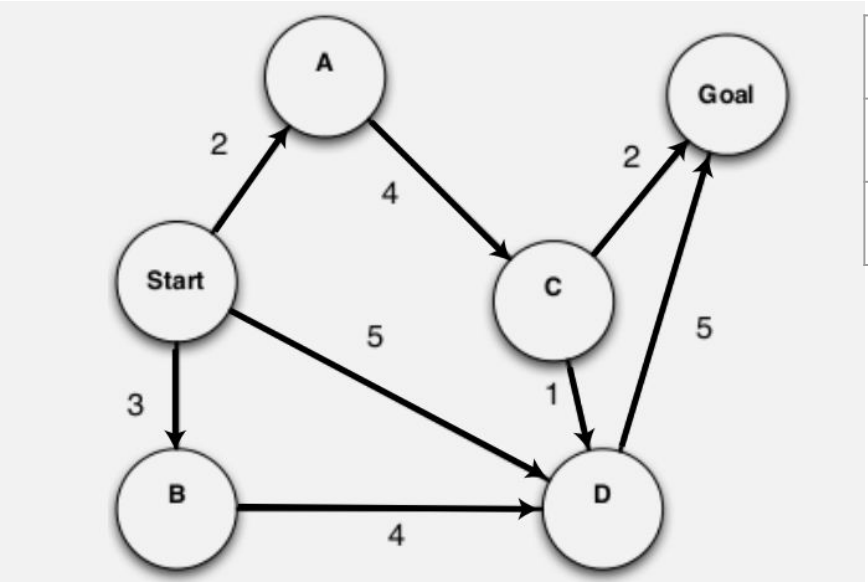


Tree Search Algorithms - BFS



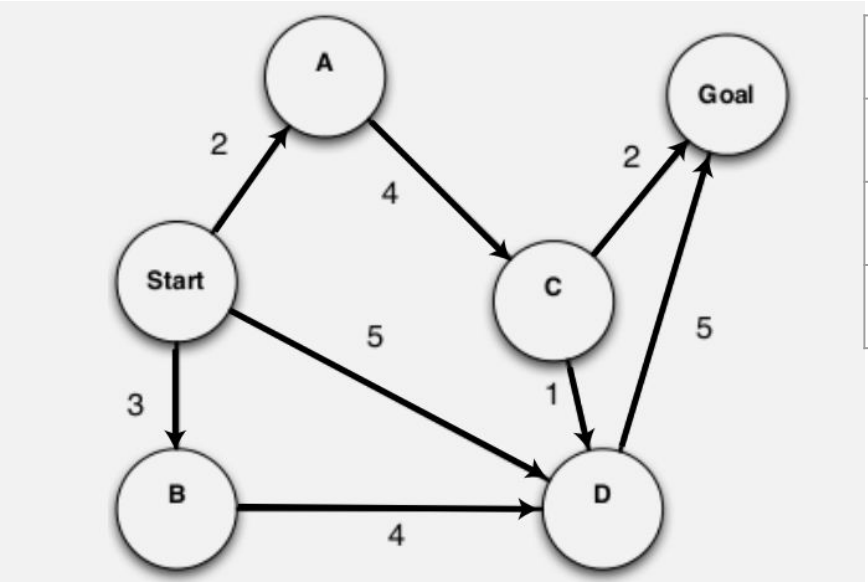
step	Frontiers/Fringe
1	**S**

Tree Search Algorithms - BFS



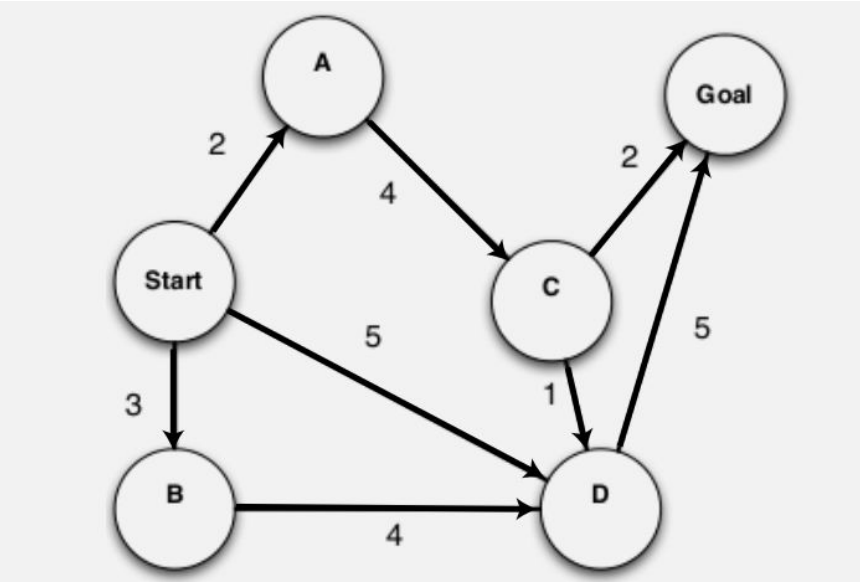
step	Frontiers
1	S
2	~~S~~, **(S A)**, (S B), (S D)

Tree Search Algorithms - BFS



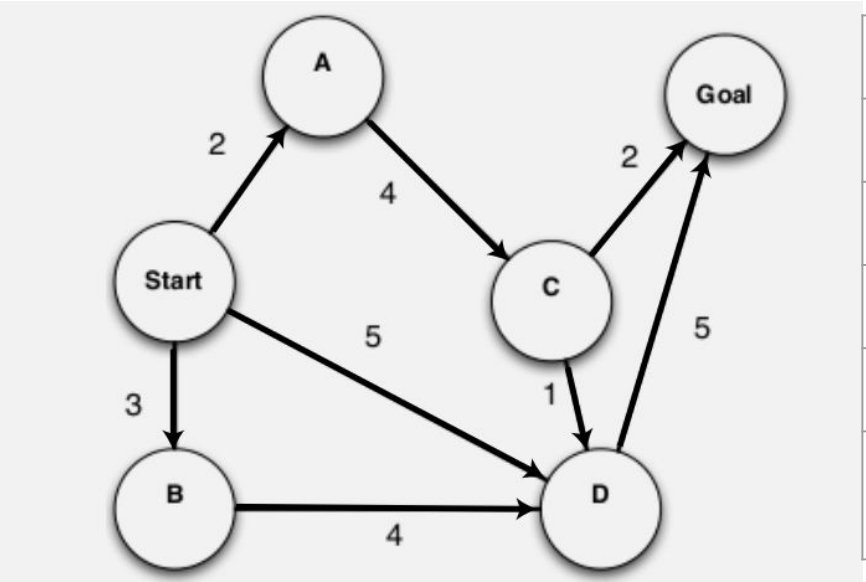
step	Frontiers
1	S
2	~~S~~, (S A), (S B), (S D)
3	~~S, (S A),~~ **(S B)**, (S D), (S A C)

Tree Search Algorithms - BFS



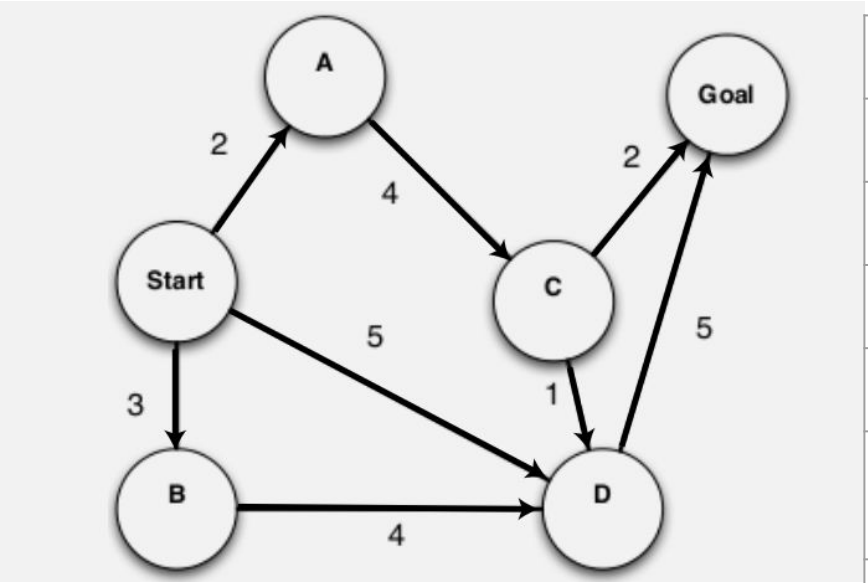
step	Frontiers
1	S
2	~~S~~, (S A), (S B), (S D)
3	~~S, (S A),~~ (S B), (S D), (S A C)
4	~~S, (S A), (S B)~~, **(S D)**, (S A C), (S B D)

Tree Search Algorithms - BFS



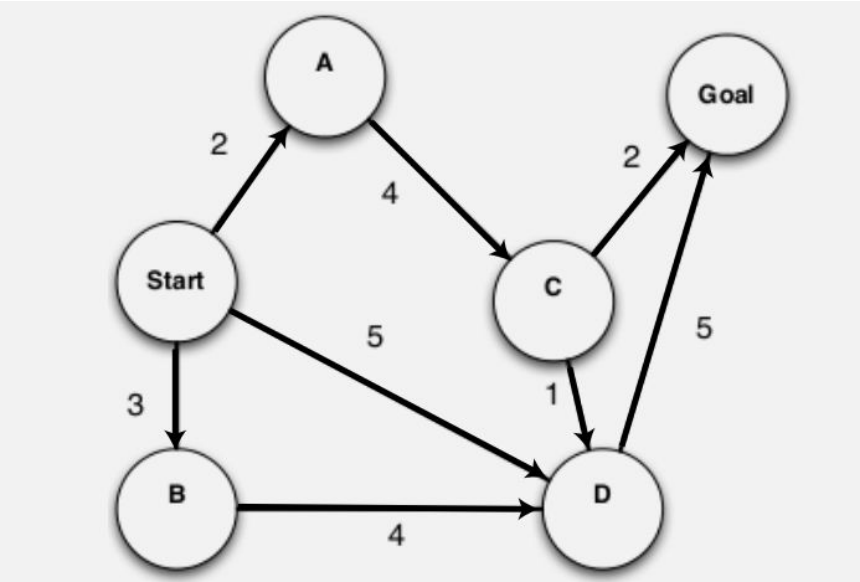
step	Frontiers
1	S
2	~~S~~, (S A), (S B), (S D)
3	~~S, (S A),~~ (S B), (S D), (S A C)
4	~~S, (S A), (S B)~~, ~~(S D)~~, (S A C)
5	~~S, (S A), (S B)~~, ~~(S D)~~, **(S A C)**, (S B D) (S D G)

Tree Search Algorithms - BFS



step	Frontiers
1	S
2	~~S~~, (S A), (S B), (S D)
3	~~S, (S A),~~ (S B), (S D), (S A C)
4	~~S, (S A), (S B)~~, ~~(S D)~~, (S A C)
5	~~S, (S A), (S B)~~, ~~(S D)~~, (S A C), (S B D) (S D G)
6	~~S, (S A), (S B)~~, ~~(S D)~~, ~~(S A C)~~, **(S B D)** (S D G) (S A C D) (S A C G)

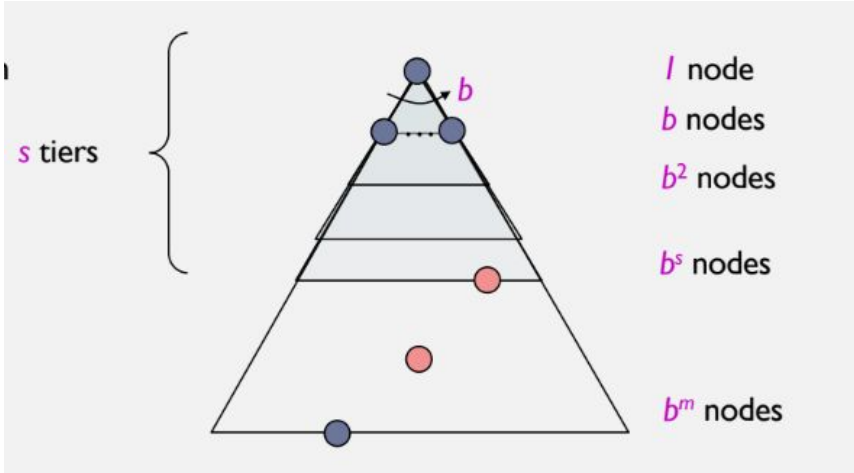
Tree Search Algorithms - BFS



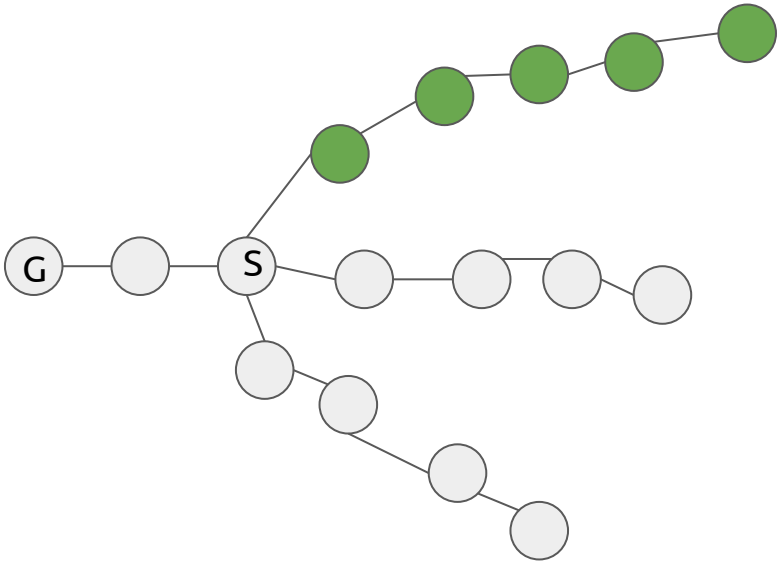
step	Frontiers
1	S
2	~~S~~, (S A), (S B), (S D)
3	~~S, (S A),~~ (S B), (S D), (S A C)
4	~~S, (S A), (S B)~~, ~~(S D)~~, (S A C)
5	~~S, (S A), (S B)~~, ~~(S D)~~, (S A C), (S B D) (S D G)
6	~~S, (S A), (S B)~~, ~~(S D)~~, ~~(S A C)~~, (S B D) (S D G) (S A C D) (S A C G)
7	~~S, (S A), (S B)~~, ~~(S D)~~, ~~(S A C)~~, ~~(S B D)~~ **(S D G)** (S A C D) (S A C G) (S B D G)

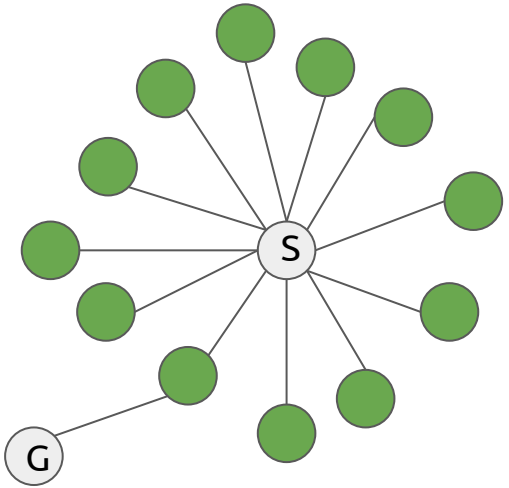
Breadth-First Search (BFS) Properties

- Time Complexity
  - Processes all nodes above shallowest solution
  - Let depth of shallowest solution be s
  - Search takes time **O(b^s)**
- Space Complexity
  - Has roughly the last tier,so **O(b^s)**
- Is it complete?
  - s must be finite if a solution exists, so yes!
- Is it optimal?
  - If costs are equal (e.g., 1)



When do DFS/BFS fall?

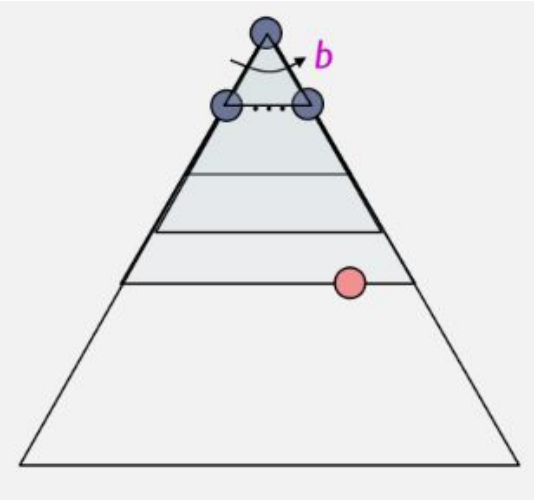




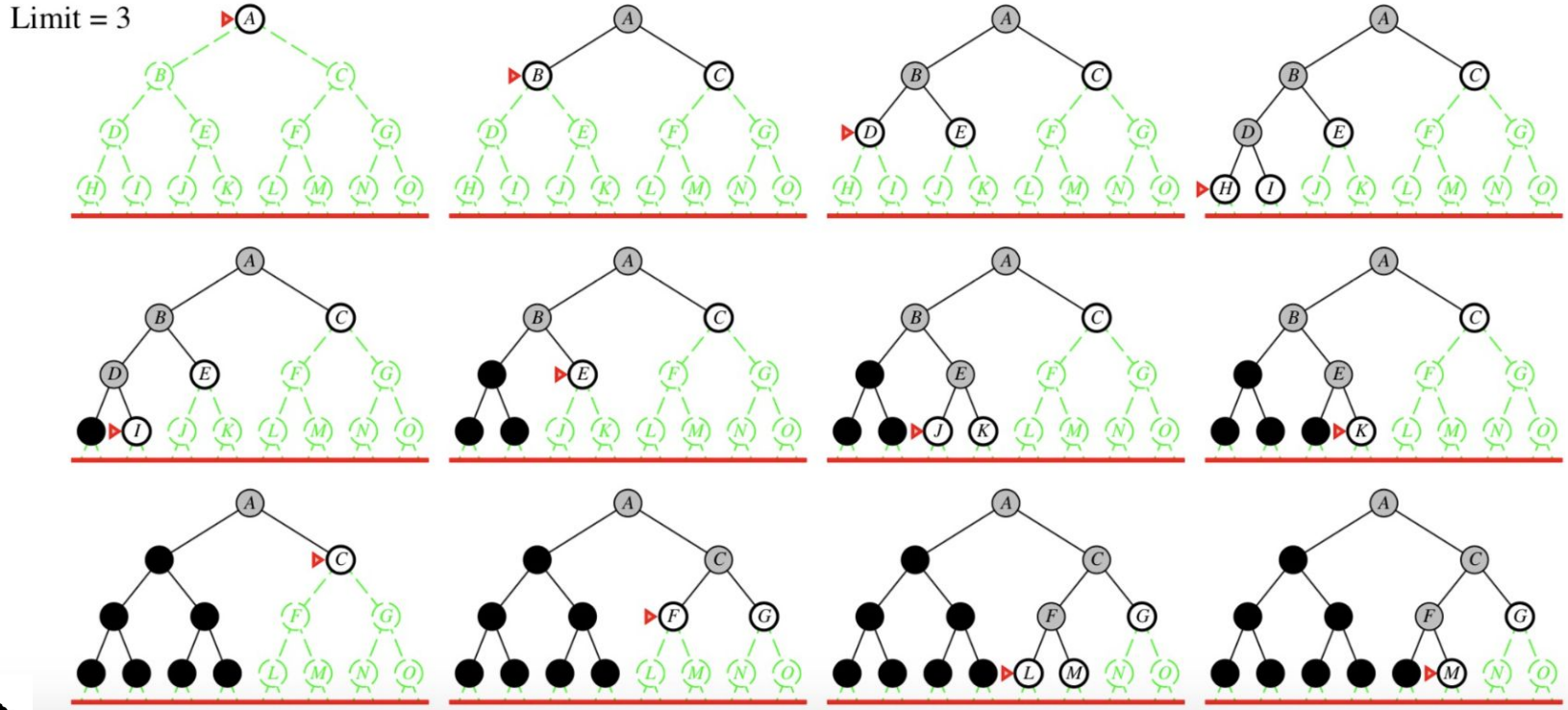
IDS = BFS + DFS

Iterative Deepening Search

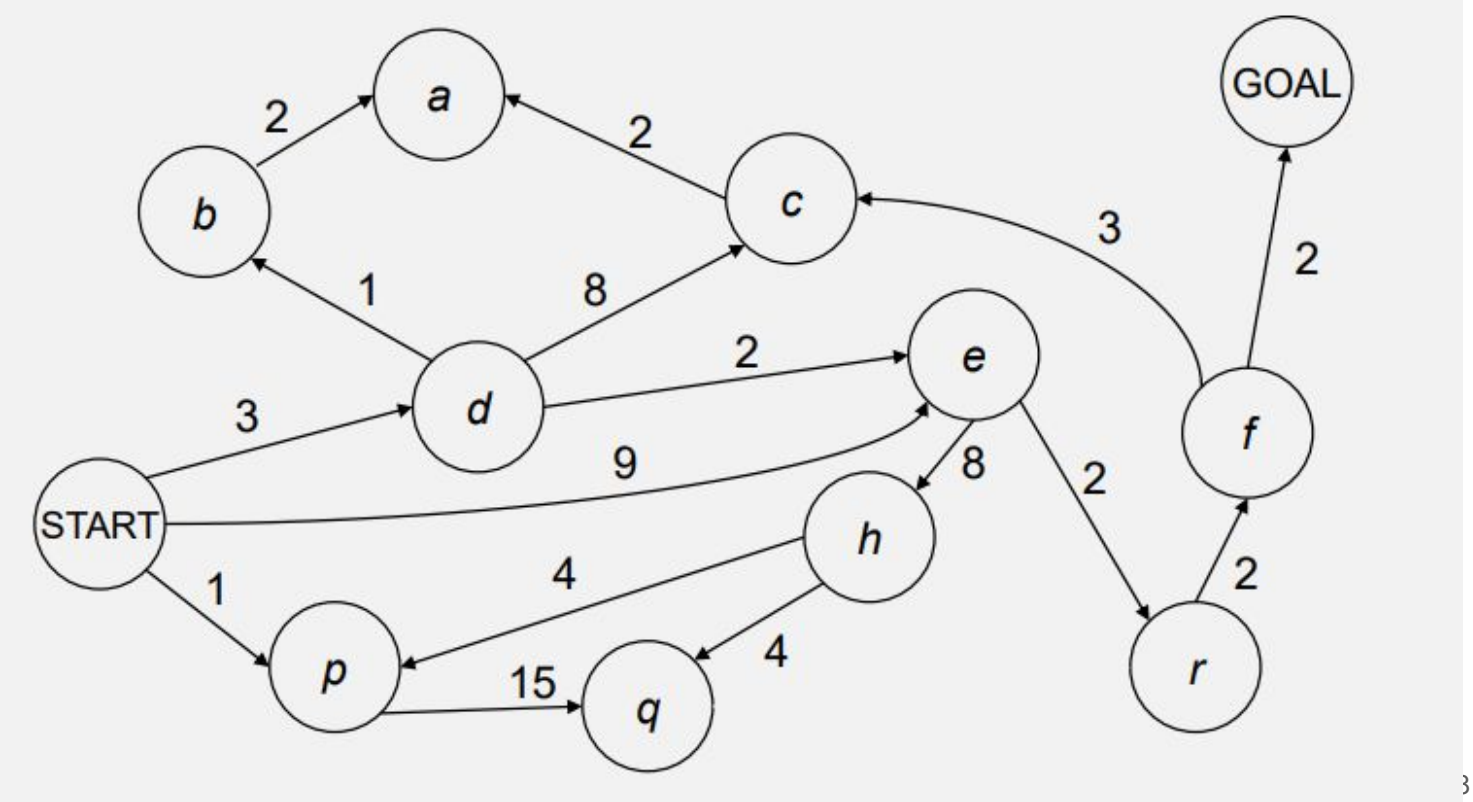
- Idea: get DFS's space advantage with BFS's time / shallow-solution advantages
  - Run a DFS with depth limit 1. If no solution…
  - Run a DFS with depth limit 2. If no solution…
  - Run a DFS with depth limit 3. .....
- Isn't that wastefully redundant?
  - Generally most work happens in the lowest level searched, so not so bad!



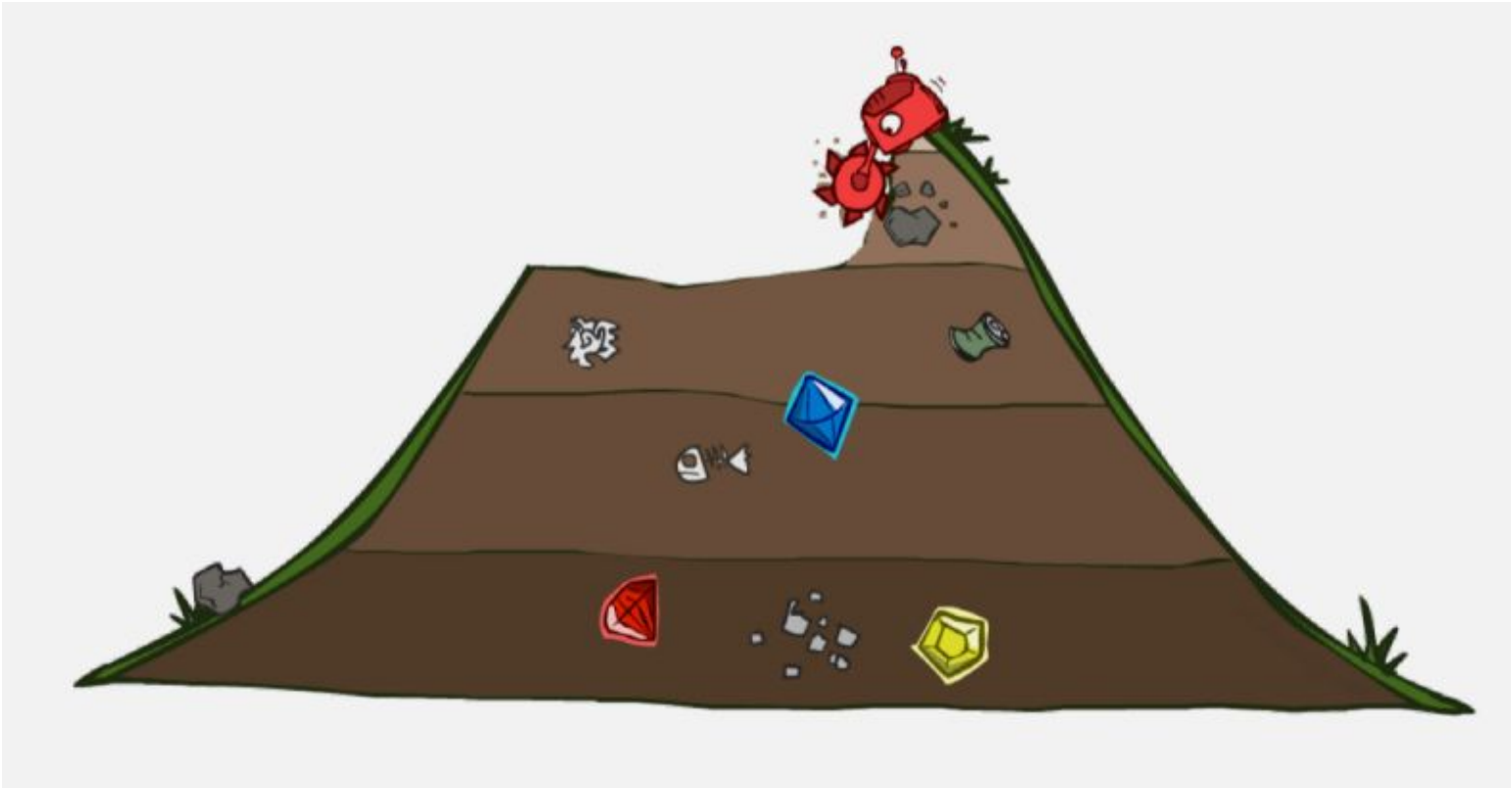
Iterative Deepening Search Example



Cost-Sensitive Search Problem

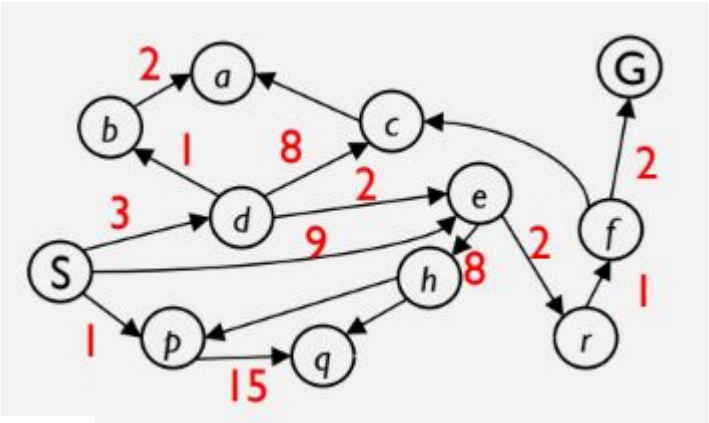


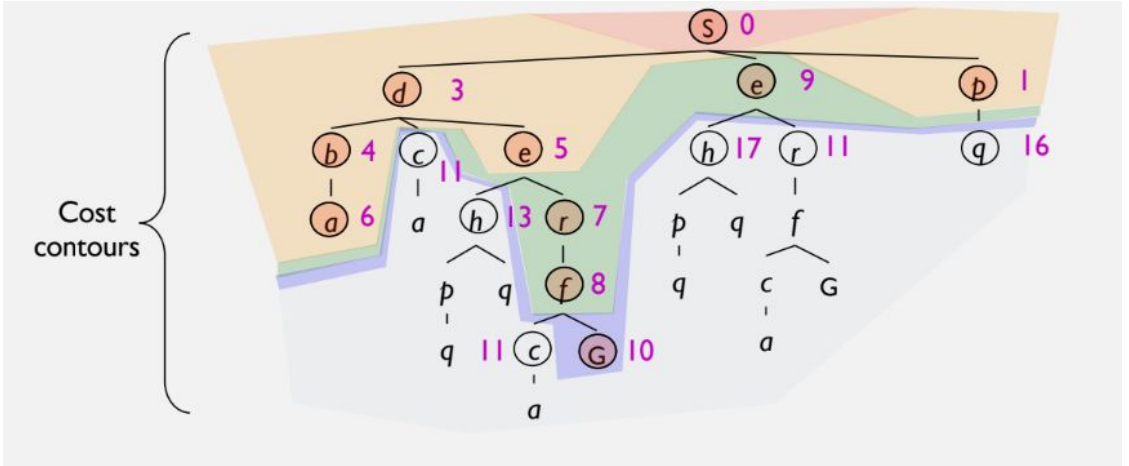
Uniform Cost Search (UCS)



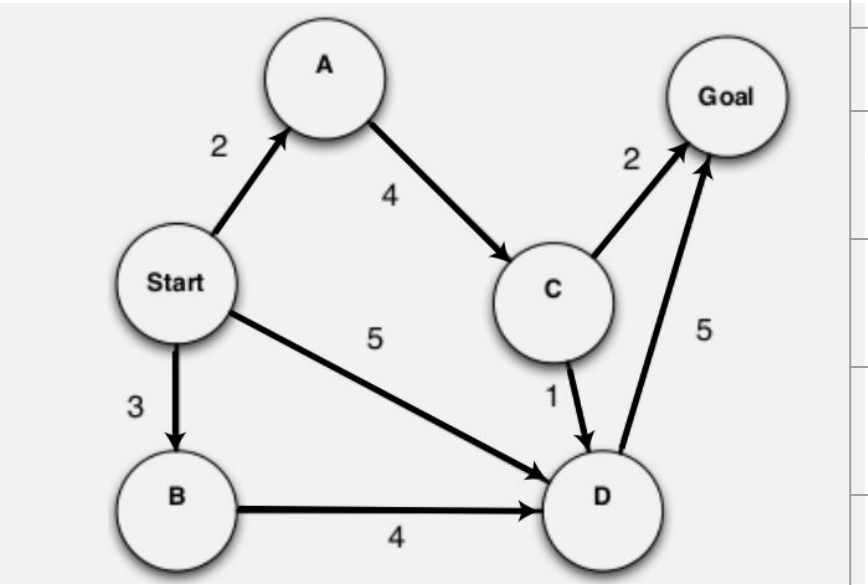
Uniform Cost Search Algorithm

- g(n) = cost from root to n
- Strategy: expand lowest g(n)
- Frontier is a priority queue sorted by g(n)



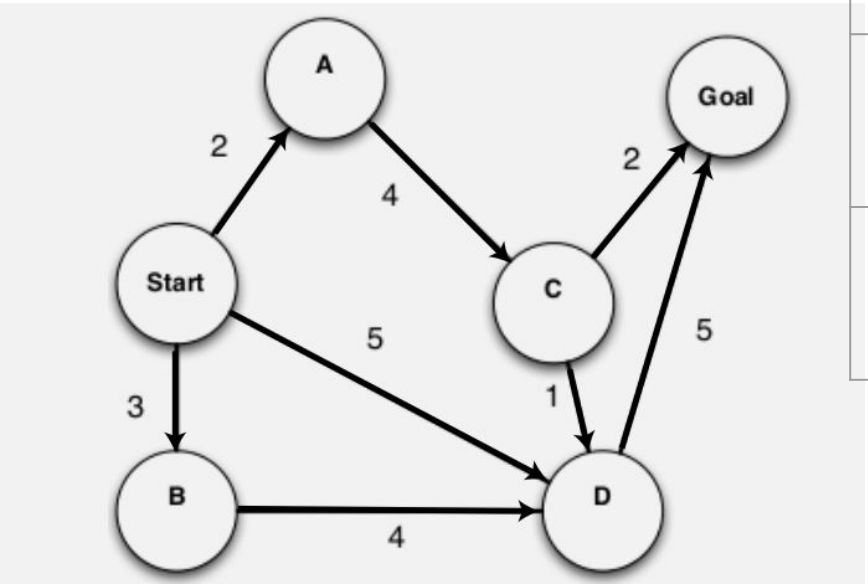


Tree Search Algorithms - UCS



step	Frontiers
1	(S 0)
2	~~(S 0)~~, ~~(S A 2)~~, (S B 3), (S D 5)
3	~~(S 0), (S A 2),~~ (S B 3), (S A C 6), (S D 5)
4	~~(S 0), (S A 2), (S B 3),~~ (S A C 6), (S B D 7), (S D 5)
5	~~(S 0), (S A 2), (S B 3),~~ (S A C 6), (S B D 7), ~~(S D 5)~~, (S D G 10)
6	~~(S 0), (S A 2), (S B 3),~~ ~~(S A C 6)~~, (S B D 7), ~~(S D 5)~~, (S D G 10), (S A C G 8), (S A C D 7)
7	~~(S 0), (S A 2), (S B 3),~~ ~~(S A C 6)~~, ~~(S B D 7)~~, ~~(S D 5)~~, (S D G 10), (S A C G 8), (S A C D 7), (S B D G 12)

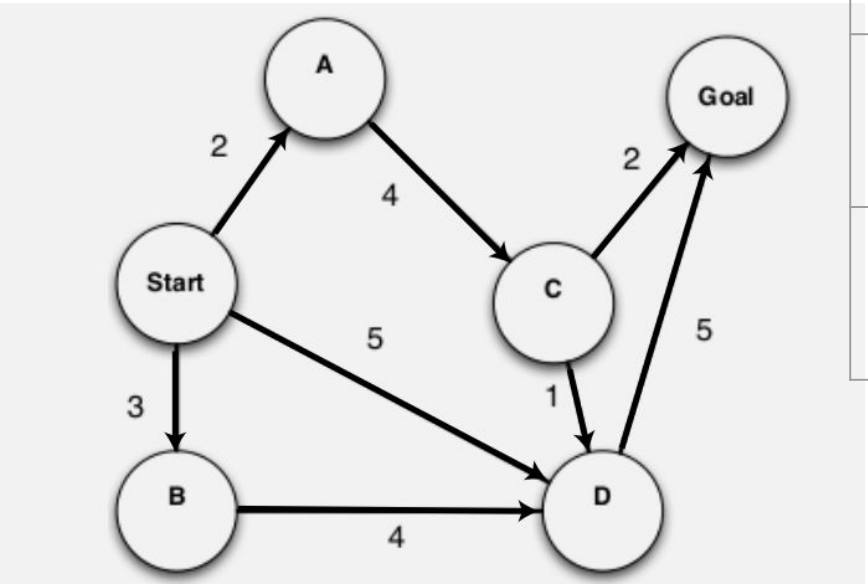
Tree Search Algorithms - UCS



step	Frontiers
7	~~(S 0), (S A 2), (S B 3),~~ ~~(S A C 6)~~, ~~(S B D 7)~~, ~~(S D 5)~~, (S D G 10), (S A C G 8), (S A C D 7), (S B D G 12)
8	~~(S 0), (S A 2), (S B 3),~~ ~~(S A C 6)~~, ~~(S B D 7)~~, ~~(S D 5)~~, (S D G 10), (S A C G 8), ~~(S A C D 7)~~, (S B D G 12), (S A C D G 12)
9	~~(S 0), (S A 2), (S B 3),~~ ~~(S A C 6)~~, ~~(S B D 7)~~, ~~(S D 5)~~, (S D G 10), **(S A C G 8)**, ~~(S A C D 7)~~, (S B D G 12), (S A C D G 12)

Question: When I See Goal in Frontiers should I Finish my search or not?

Tree Search Algorithms - UCS

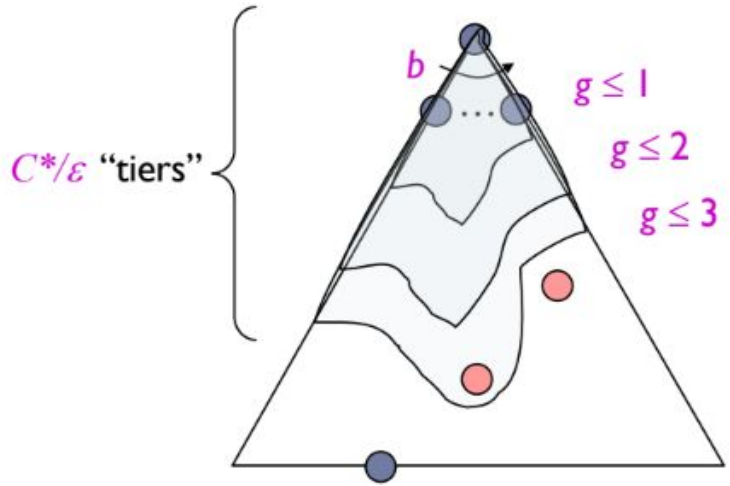


step	Frontiers
7	~~(S 0), (S A 2), (S B 3),~~ ~~(S A C 6)~~, ~~(S B D 7)~~, ~~(S D 5)~~, (S D G 10), (S A C G 8), (S A C D 7), (S B D G 12)
8	~~(S 0), (S A 2), (S B 3),~~ ~~(S A C 6)~~, ~~(S B D 7)~~, ~~(S D 5)~~, (S D G 10), (S A C G 8), ~~(S A C D 7)~~, (S B D G 12), (S A C D G 12)
9	~~(S 0), (S A 2), (S B 3),~~ ~~(S A C 6)~~, ~~(S B D 7)~~, ~~(S D 5)~~, (S D G 10), **(S A C G 8)**, ~~(S A C D 7)~~, (S B D G 12), (S A C D G 12)

Question: When I See Goal in Frontiers should I Finish my search or not? **NO You Must Finish When Select Goal for Expand**

Uniform Cost Search (UCS) Properties

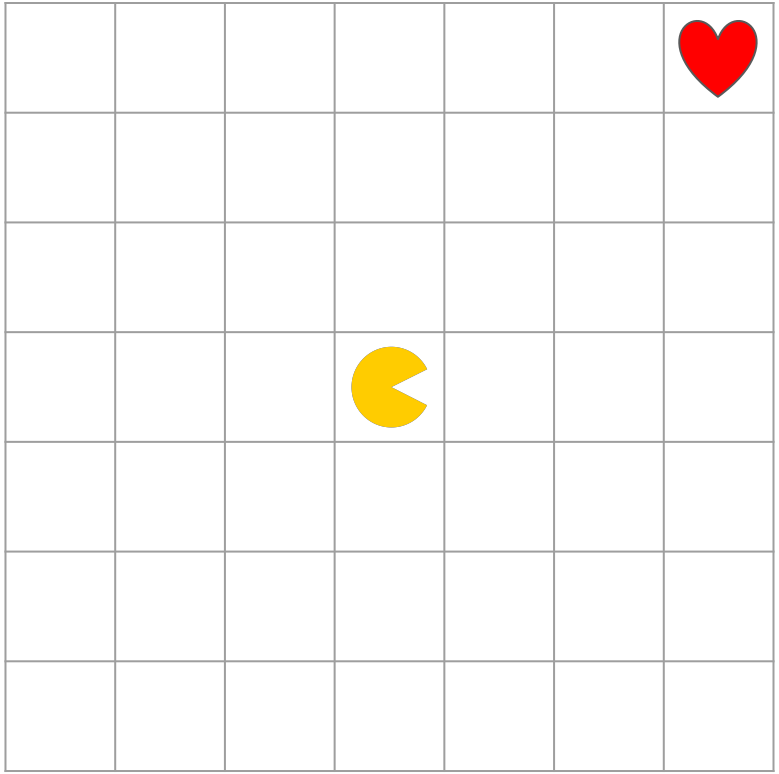
- Time Complexity
  - Expands all nodes with cost less than cheapest solution!
  - If that solution costs C* and arcs cost at least e , then the “effective depth” is roughly C*/e
  - O(b^(C*/e))
- Space Complexity
  - Has roughly the last tier,so O(b^(C*/e))
- Is it complete?
  - Assuming C* is finite and e > 0, yes!
- Is it optimal?
  - Yes!



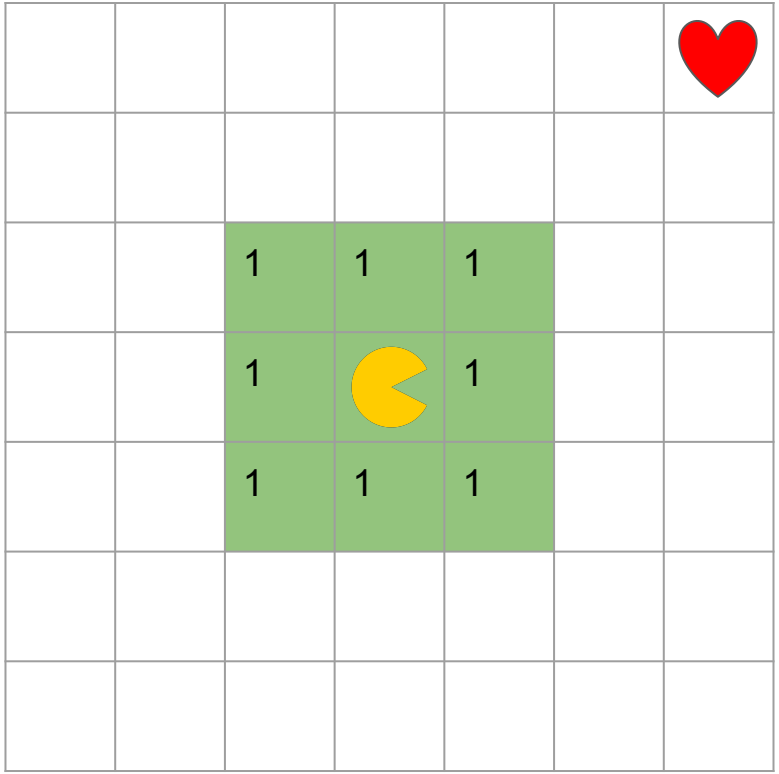
Recap: Uninformed Search Algorithms

- **Breadth-First Search (BFS):**
  - Dequeues nodes in order of shallowest depth first (FIFO queue).
- **Depth-First Search (DFS):**
  - Dequeues the most recently added node first (LIFO stack).
- **Iterative Deepening Search (IDS):**
  - Repeatedly runs DFS, increasing the depth limit each time.
- **Uniform Cost Search (UCS):**
  - Dequeues the node with the lowest cumulative path cost (g(n)) first.

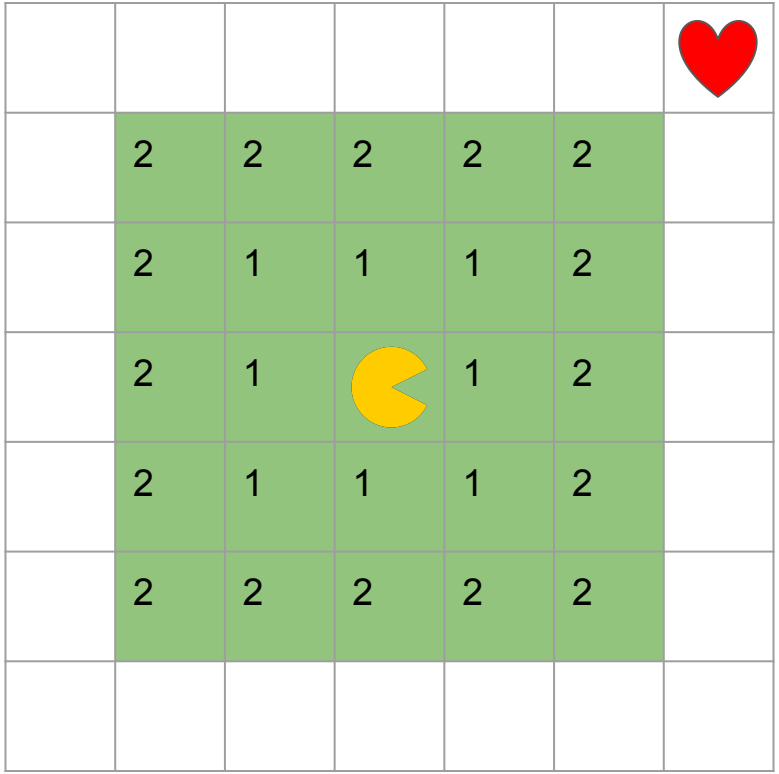
Why UCS is not Enough for us?



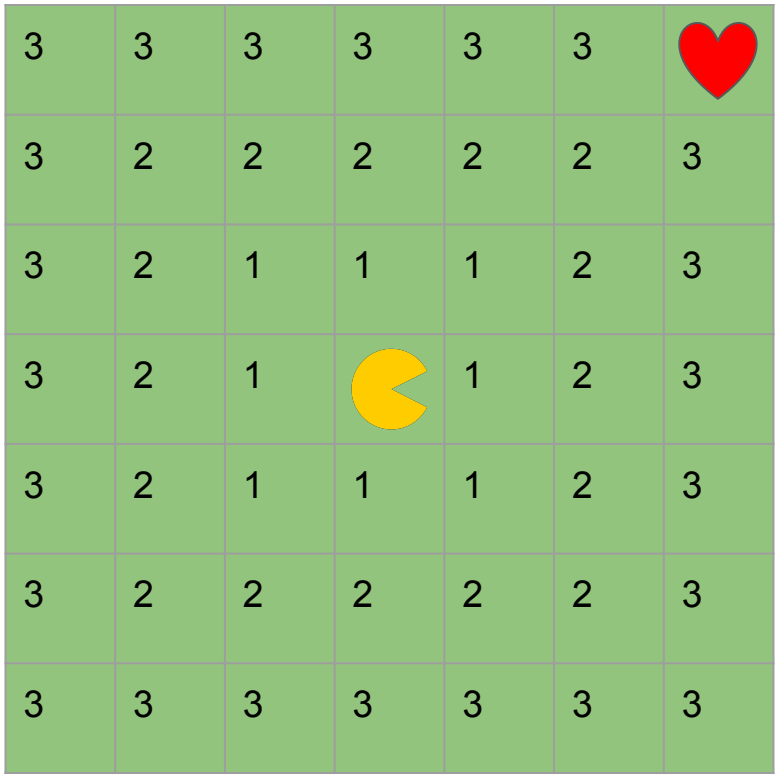
Why UCS is not Enough for us?



Why UCS is not Enough for us?

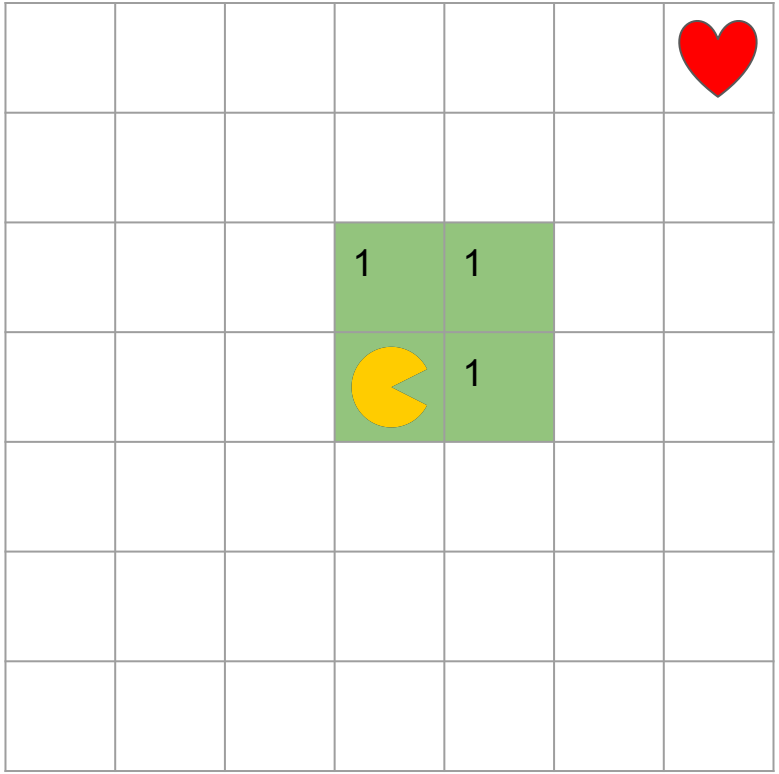


Why UCS is not Enough for us?

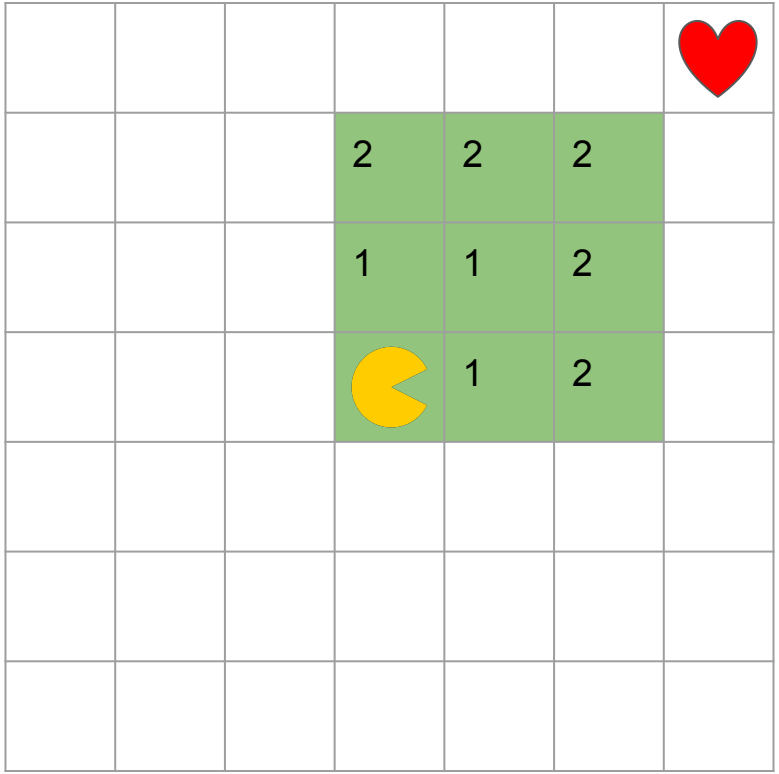


No information about goal location!

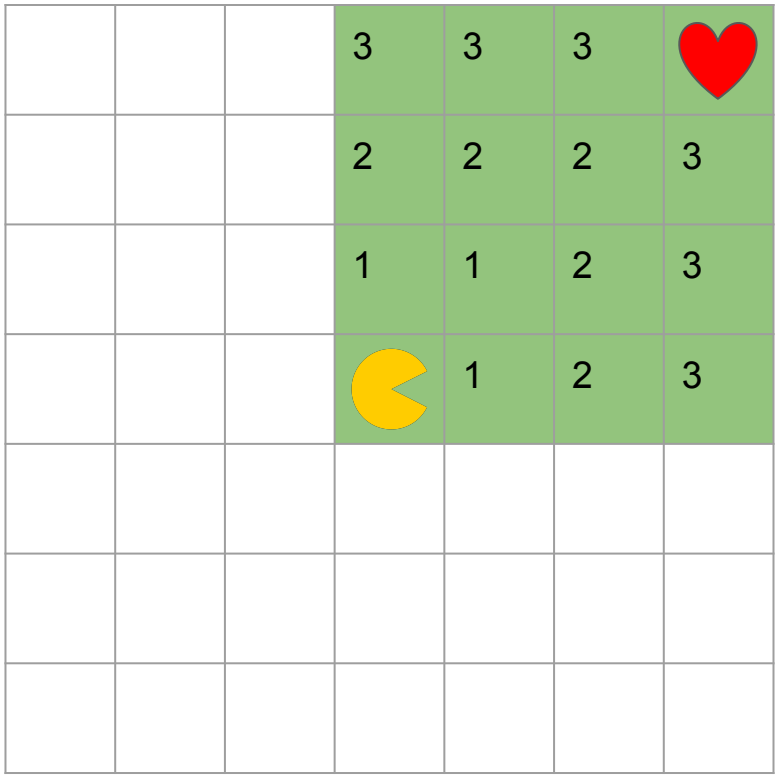
What does our intuition do?



What does our intuition do?

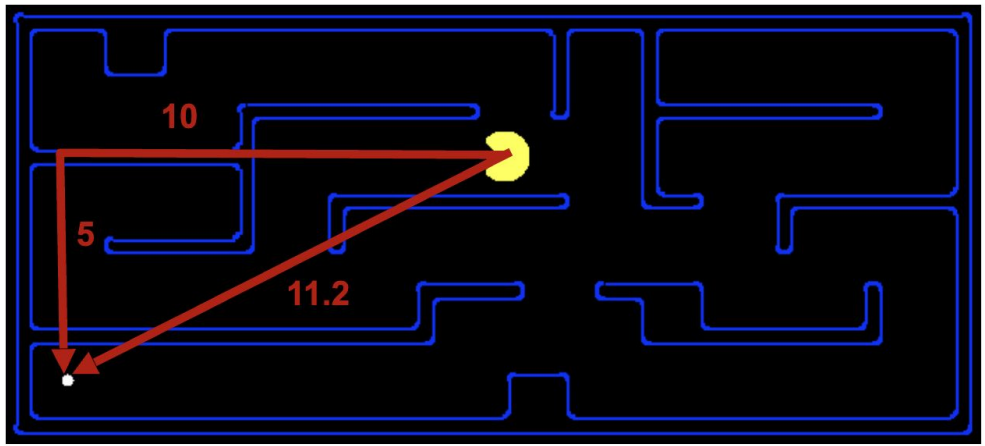


What does our intuition do?



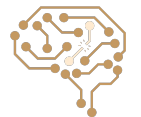
Search Heuristics

- A function h(s): **estimates** how close a state is to a goal
- For each search problem we design/consider particular heuristics
- Example:
  - Manhattan distance
  - Euclidean distance



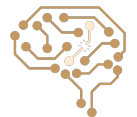
Search Heuristic

- Particular heuristics for different search problems
- What if the goal state changes?
- What if the start state changes?
- What if the cost between states changes?



Informed Search Algorithms

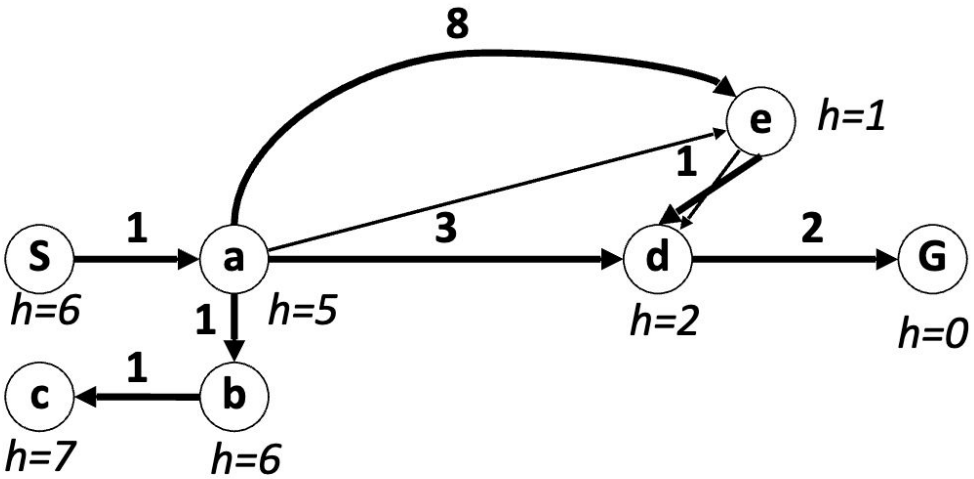
- Greedy
- A^*

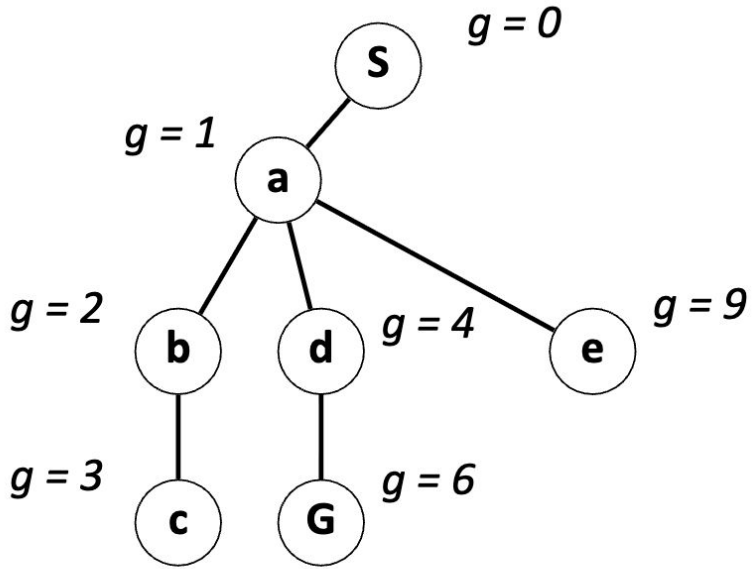


Greedy Search

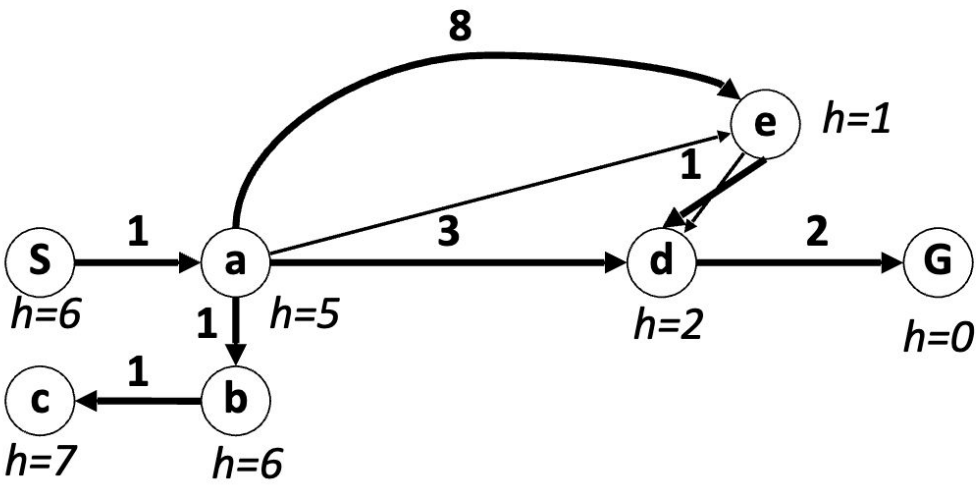
- Expand the node that have the best heuristic (Best First Search)

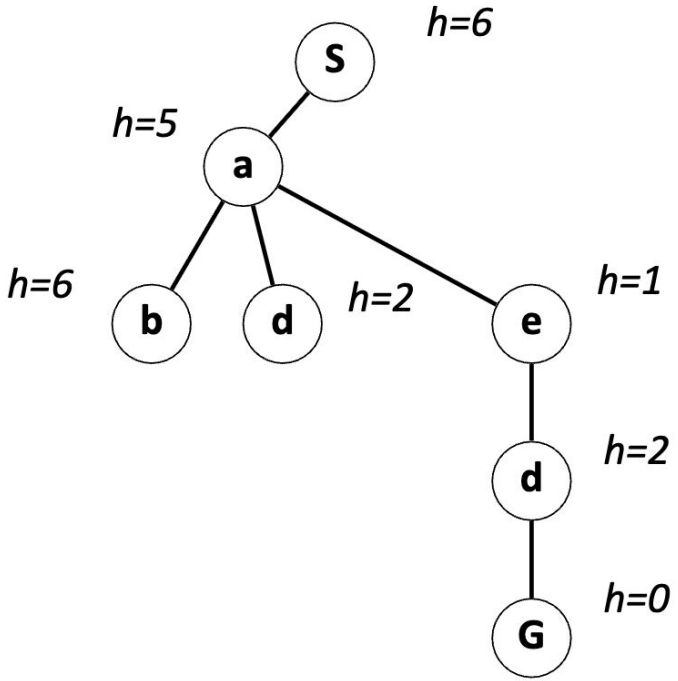
UCS Example





Greedy Example

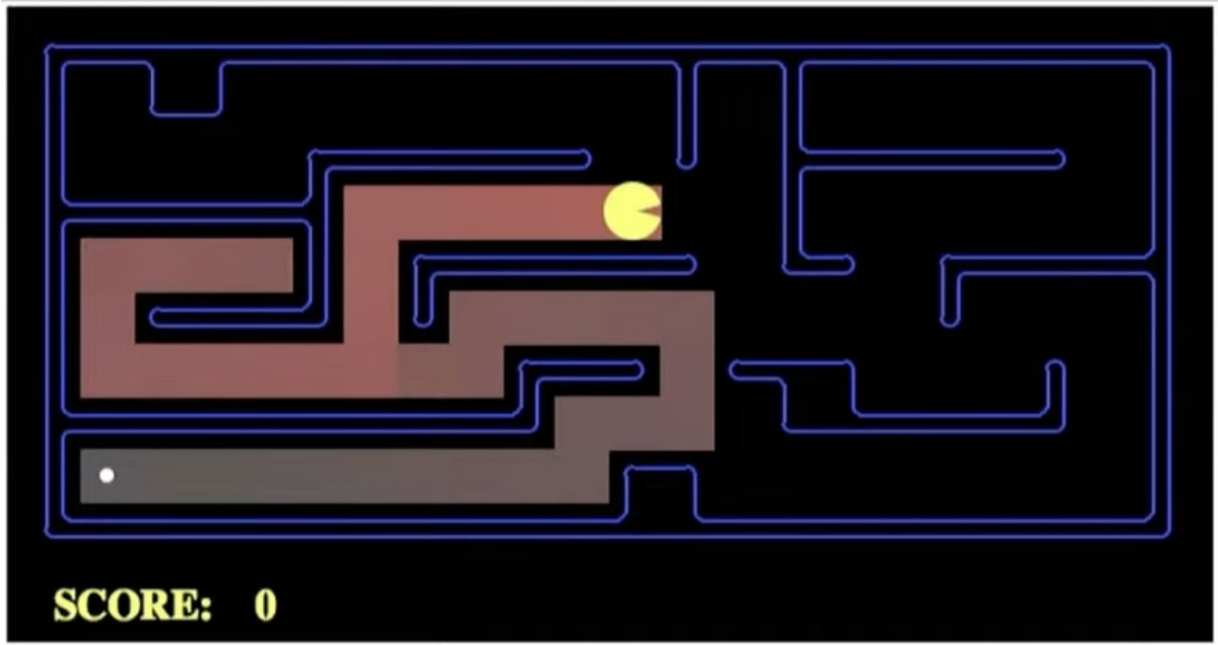


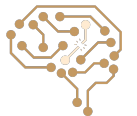


Greedy Search Characteristics

- Greedy Best-first search is sub-optimal
- Worst Case: Like DFS, it may explore everything
- Completeness (like DFS):
  - With cycle checking
  - Finite number of states

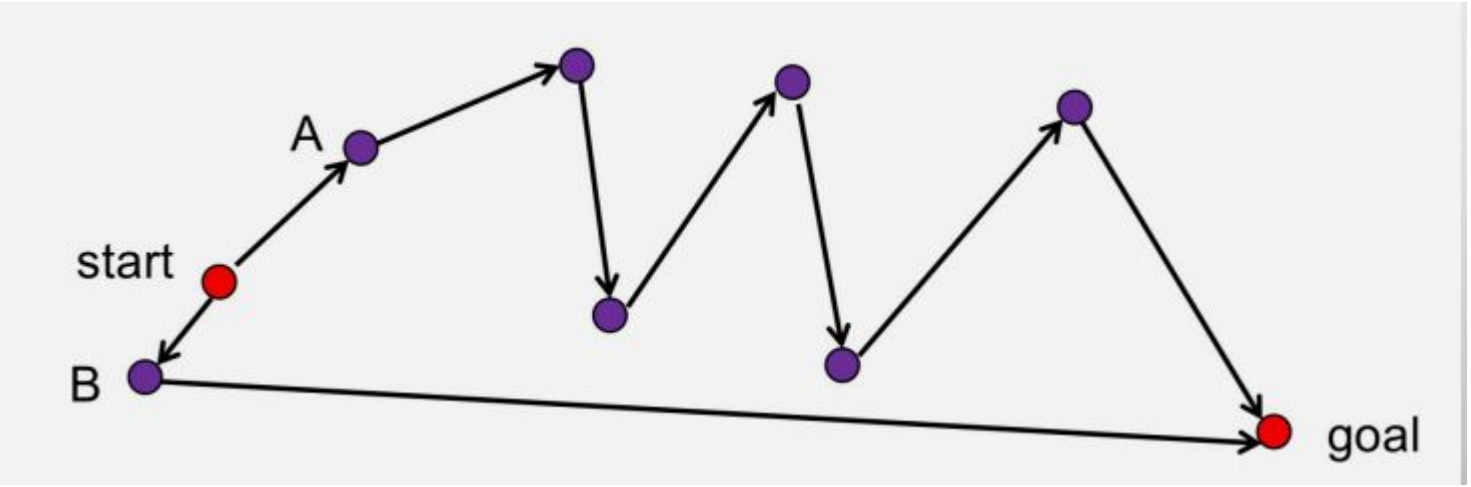
How Can It Go Wrong? - Pacman Small Maze





How Can It Go Wrong?

- Heuristic: Expand the node that seems closest...



Next Session: A*